WhongNaiMor Project Report

By Sysiphus





Team Members

Thanwisit Angsachon 682115018
Parinthorn Saengwiman 682115025
Natphisit Pintha 682115015

Propose To

Asst. Prof. Dr. Pree Thiengburanathum

This report is part of the term project of Database System and Database System Design course (953212) class of 1/2569

Introduction

Business background

WhongNaiMor is a social media platform inspired by Facebook and designed specifically for the Chiang Mai University (CMU) community, especially students and personnel. The platform allows users to share stories, interesting experiences, questions, opinions, recommendations, and information with other members of the community. It also provides a way for students to discover new people and exchange different perspectives. Unlike a general social media platform, WhongNaiMor focuses on creating a community where CMU members can interact in a more relevant and controlled environment.

The idea was inspired by the Facebook group “ลูกช้าง มช.”, which is widely used to share information about CMU and its surrounding areas. However, several problems can occur because the group is hosted on a public platform. Posts and comments may expose sensitive information to people outside the university, while access to the group may not be sufficiently restricted. There has also been an incident where an unrelated person gained control of the group, which affected its credibility and trustworthiness. Furthermore, important announcements, warnings, or emergency posts can easily be buried under popular or controversial content. Although anonymous posting and community action have been used to address some issues, the problem of important information being overlooked remains unresolved.

Business problem

Today, CMU students share information, experiences, questions, and recommendations across different social media platforms. Without a proper database, important information is difficult to organize and manage. User information, posts, comments, reactions, and reports may be stored separately or become difficult to track. This creates a problem when students want to find specific information. For example, an important announcement or a useful recommendation can quickly disappear because many new posts are added every day. Students may also have difficulty knowing whether the information they find is reliable or relevant to the CMU community.

Another problem is content management and moderation. Without a structured database, it is difficult for administrators to track reported posts, identify inappropriate content, and manage important posts. Questions, lost-and-found posts, warnings, and announcements may not receive enough attention because they are mixed with normal posts. Administrators also cannot easily answer questions such as **“Which posts are reported most often?”** or **“Which important posts need more attention?”** As the community grows, managing this information manually becomes more difficult and time-consuming. Therefore, WhongNaiMor needs a proper database to organize community information and make it easier for both students and administrators to manage and find useful content.

Motivation

A database is the right solution because it can store and organize users, posts, comments, reactions, reports, and post categories in one structured system. With this database, WhongNaiMor can connect related information and make it easier to search, update, and manage. Administrators will be able to track reported content, manage inappropriate posts, and identify important posts more efficiently. Students will also be able to find relevant information without searching through many different platforms. The system can prioritize questions, lost-and-found posts, warnings, and announcements so that important information is less likely to be missed. In the future, the database can also help the team understand how students use the platform and improve the community based on real data. **Overall, the database will make WhongNaiMor more organized, manageable, and useful for the CMU community.**

Users of the system

User role	Main tasks
CMU User	Log in, create posts, comment, react, ask questions, and share experiences.
Non-CMU User	Browse and read public posts about CMU student life.
Moderator	Manage users, review reported posts, and remove inappropriate content.

Objectives and scope

Objectives

#	Objective
O1	Create a student-centered community platform for CMU students.
O2	Enable secure access through CMU accounts.
O3	Highlight important posts for easier discovery by filtering inappropriate content and prioritizing them..
04	Allow users to share experiences, recommendations, questions, and useful information.

Scope

In scope (the database WILL cover)	Out of scope (the database will NOT cover)
User accounts and CMU account information	Mobile application data
Posts, images, comments, and reactions	External social media data
Reports and inappropriate-content records	Direct messages between users
Notification	No account creation and personalization. (Use template accounts)

Business rules

#	Business rule	Category
BR1	A user may write zero or more posts, but each post is written by exactly one user.	**Relationship rules**
BR2	A post may have zero or more comments, but each comment belongs to exactly one post.	**Relationship rules**
BR3	A user may write zero or more comments, but each comment is written by exactly one user.	**Relationship rules**
BR4	A comment may have zero or more replies, but each reply answers exactly one parent comment on the same post.	**Relationship rules**
BR5	A post may contain zero or more media files, but each media file belongs to exactly one post.	**Relationship rules**
BR6	A post may belong to many categories, and a category may hold many posts.	**Relationship rules**
BR7	A user may follow many categories, and a category may be followed by many users.	**Relationship rules**
BR8	A user may react to many posts, and a post may receive reactions from many users.	**Relationship rules**
BR9	A user may react to many comments, and a comment may receive reactions from many users.	**Relationship rules**
BR10	A user may report many posts, and a post may be reported by many users.	**Relationship rules**
BR11	A user may receive many notifications, but each notification is sent to exactly one user and refers to exactly one post.	**Relationship rules**
BR12	A moderator may review many reports, but each report is reviewed by at most one moderator.	**Relationship rules**
BR13	A moderator may publish many announcements, but each announcement is published by exactly one moderator.	**Relationship rules**
BR14	A moderator may hide many posts, but each post is hidden by at most one moderator.	**Relationship rules**
BR15	A moderator may record many moderation log entries, but each log entry is recorded by exactly one moderator.	**Relationship rules**
BR16	Every post must belong to at least one category.	**Participation rules**
BR17	Every media file must belong to a post and is deleted when that post is deleted.	**Participation rules**
BR18	Every comment must belong to a post and is deleted, together with its replies and reactions, when that post is deleted.	**Participation rules**
BR19	A reply must have a parent comment, a top-level comment has no parent, and deleting a comment also deletes all of its replies.	**Participation rules**
BR20	Every reaction must belong to exactly one user and to exactly one post or comment, and is deleted when that post or comment is deleted.	**Participation rules**
BR21	A report does not need a reviewer while it is Open, but it must have one once it is Reviewed or Dismissed.	**Participation rules**

BR22	A moderation log entry does not have to point to an existing post, so it is kept even after the post it mentions is deleted.	**Participation rules**
BR23	Each user has a unique CMU email address ending in @cmu.ac.th.	**Attribute / constraint rules**
BR24	A user's display name must be 2 to 50 characters long.	**Attribute / constraint rules**
BR25	A user may react only once to the same post and only once to the same comment.	**Attribute / constraint rules**
BR26	A reaction type must be one of like, love, haha, sad, angry, or care.	**Attribute / constraint rules**
BR27	Each category has a unique name and a priority of 1 (most important), 2, or 3.	**Attribute / constraint rules**
BR28	A category cannot be deleted while any post still belongs to it.	**Attribute / constraint rules**
BR29	A post must have a title of at most 120 characters and content of at most 5,000 characters.	**Attribute / constraint rules**
BR30	A comment must contain 1 to 1,000 characters.	**Attribute / constraint rules**
BR31	A post may have at most 10 images and 1 video, and each file can be at most 50 MB.	**Attribute / constraint rules**
BR32	A media type must be either image or video.	**Attribute / constraint rules**
BR33	Each report has exactly one status: Open, Reviewed, or Dismissed.	**Attribute / constraint rules**
BR34	A report must give a reason of at most 300 characters.	**Attribute / constraint rules**
BR35	A user may have only one Open report on the same post at a time.	**Attribute / constraint rules**
BR36	A comment's creation time cannot be earlier than its post's creation time.	**Attribute / constraint rules**
BR37	Only moderators may review reports, hide or unhide posts, edit or delete other users' posts, and manage categories and user roles.	**Attribute / constraint rules**
BR38	There must always be at least one moderator, and a moderator cannot change their own role.	**Attribute / constraint rules**
BR39	A hidden post is visible only to its author and moderators and cannot receive new comments or reactions.	**Attribute / constraint rules**
BR40	When a post is edited, the time of the edit and the user who made it are recorded.	**Attribute / constraint rules**
BR41	When a post is published, every user who follows one of its categories, except its author, receives one notification about it.	**Attribute / constraint rules**
BR42	A post's reaction count, comment count, and report count, and a category's follower count, are derived from other records and are not stored.	**Attribute / constraint rules**
BR43	A user has a UserID, CMUAccount, DisplayName, Email, Role (isCMUP, isMod), and CreatedAt.	**Attribute / constraint rules**
BR44	A post has a PostID, UserID, Title, Content, isImportant, CreatedAt, UpdatedAt, Edited (EditedAt, EditedBy), and Hidden (isHidden, HiddenAt, HiddenBy, HiddenReason).	**Attribute / constraint rules**
BR45	A comment has a CommentID, PostID, UserID, ParentCommentID, Content, and CreatedAt.	**Attribute / constraint rules**
BR46	A category has a CategoryID, CategoryName, Description, and Priority.	**Attribute / constraint rules**
BR47	A media file has a MediaID, PostID, FileURL, and MediaType.	**Attribute / constraint rules**
BR48	A post reaction has a ReactionID, PostID, UserID, ReactionType, and CreatedAt.	**Attribute / constraint rules**
BR49	A comment reaction has a ReactionID, CommentID, UserID, ReactionType, and CreatedAt.	**Attribute / constraint rules**
BR50	A user interest has a UserInterestID, UserID, CategoryID, and CreatedAt.	**Attribute / constraint rules**
BR51	A post category has a PostCategoryID, PostID, CategoryID, and CreatedAt.	**Attribute / constraint rules**

BR52	A report has a ReportID, PostID, UserID, Reason, Status, ReviewedBy, ReviewedAt, and CreatedAt.	**Attribute / constraint rules**
BR53	A notification has a NotificationID, UserID, PostID, Message, IsRead, and CreatedAt.	**Attribute / constraint rules**
BR54	A moderation log entry has a LogID, ActorID, Action, PostID, PostTitle, TargetID, Detail, and CreatedAt.	**Attribute / constraint rules**
BR55	A user may be CMU Personnel (isCMUP = True), a Moderator (isMod = True), both, or neither, as students are.	**Specialisation rules (EER)**
BR56	A user can be CMU Personnel and a Moderator at the same time, and every Moderator must also be CMU Personnel.	**Specialisation rules (EER)**
BR57	A post is either an Announcement (isImportant = True) or a Normal Post (isImportant = False), never both.	**Specialisation rules (EER)**

EER Diagram

*The EER diagram might already reflect 3NF.

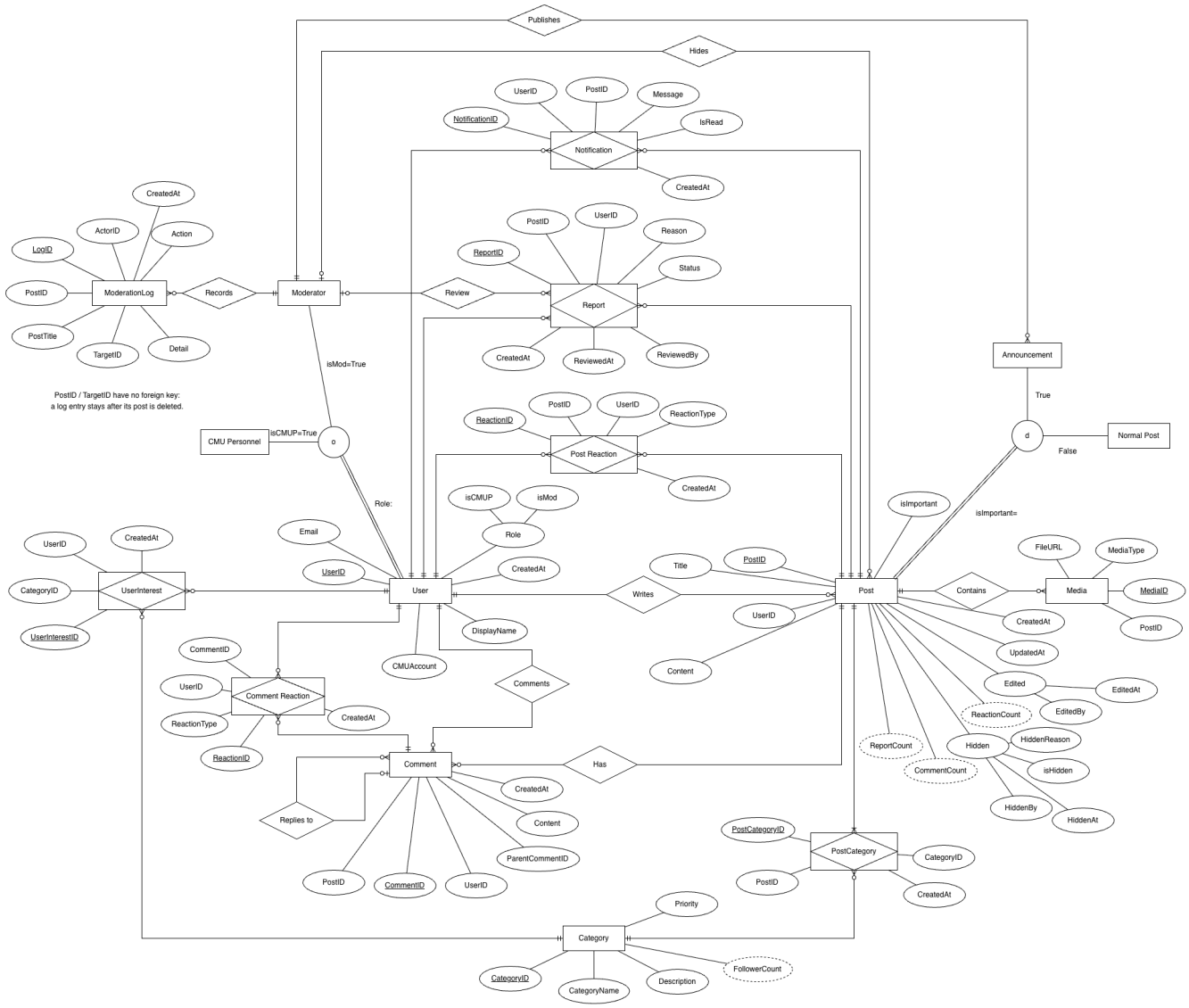
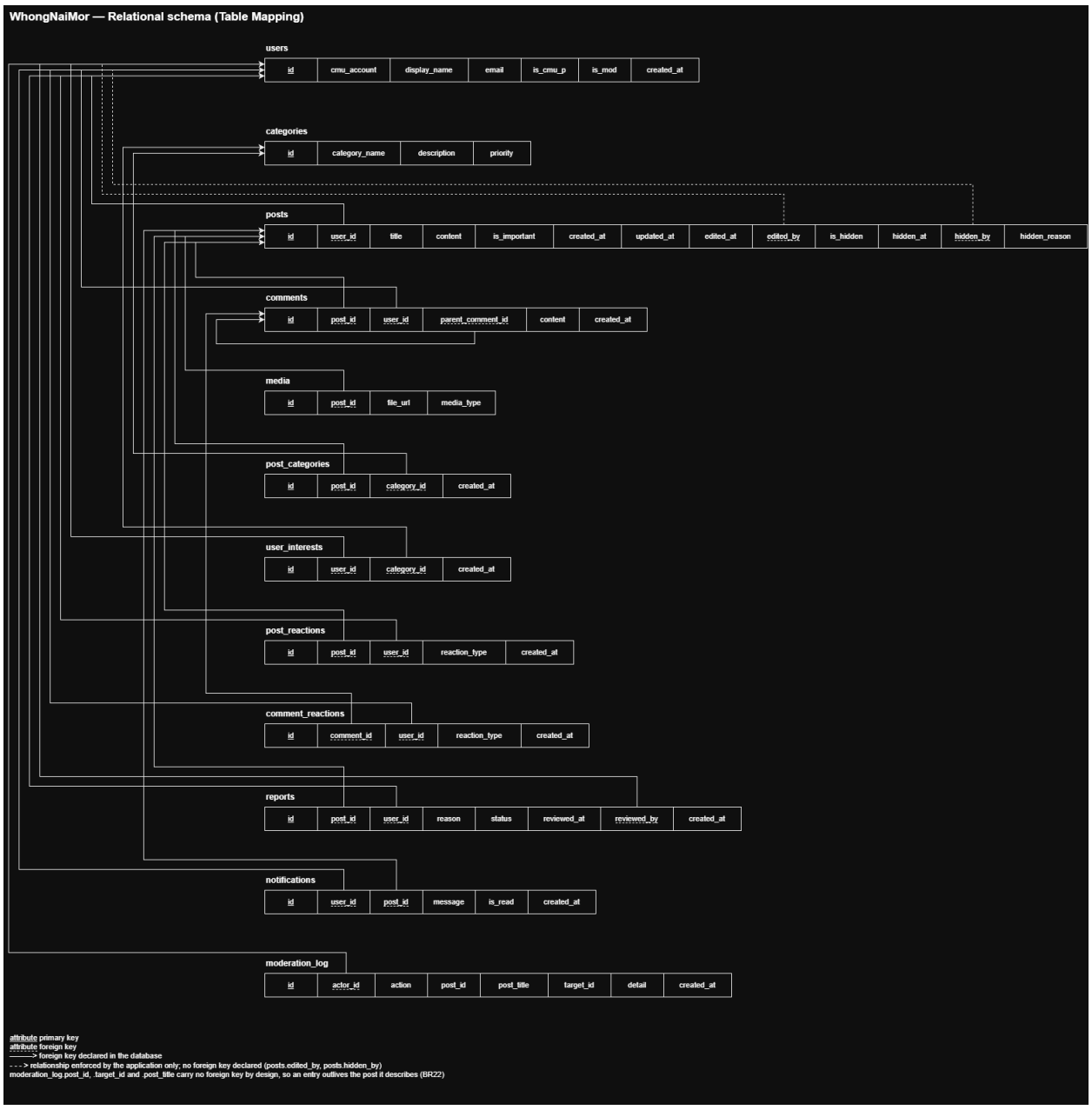


Table Mapping



*According to the EER diagram, the schemas are already in 3NF, so the further normalization steps are more like what-if situations. (Step by step mapping are on the next pages)

TABLE MAPPING · STEP 1 OF 10

Strong entity → one table

Rule

Each strong entity becomes a table with its simple attributes; its key becomes the primary key.

In our EER

USER, with the key UserID and the attributes CMUAccount, DisplayName, Email, CreatedAt and the role flags isCMUP and isMod.

In the tables

USERS(**id**, cmu_account, display_name, email, is_cmu_p, is_mod, created_at)

users

id	cmu_account	display_name	email	is_cmu_p	is_mod	created_at

categories

id	category_name	description	priority

moderation_log

id	actor_id	action	post_id	post_title	target_id	detail	created_at

Six tables come from this step; each key is a uuid of its own

```
create table public.users (
  id            uuid        not null default gen_random_uuid(),
  cmu_account   text        not null,
  display_name  text        not null,
  email         text        not null,
  is_cmu_p      boolean     not null default false,
  is_mod        boolean     not null default false,
  created_at    timestamptz not null default now(),
  constraint users_pkey primary key (id),
  constraint users_cmu_account_key unique (cmu_account),
  constraint users_email_key unique (email),
  constraint chk_cmu_account_domain
    check (cmu_account ~* '^[a-zA-Z0-9._%+-]+@cmu\.ac\.th$'),
  constraint chk_mod_requires_cmup check (is_mod = false or is_cmu_p = true)
);
```

Why it matters

Six tables come from this step: users, categories, posts, comments, media and moderation_log. Every key is a uuid made by gen_random_uuid(), so the API and the seed scripts can create rows without asking the database for the next number. cmu_account and email are UNIQUE: two more candidate keys. The CHECK accepts only @cmu.ac.th accounts (BR23).

TABLE MAPPING · STEP 2 OF 10

Weak entity → table with the owner's key

Rule

A weak entity becomes a table whose primary key is the owner's key plus its own partial key.

In our EER

We have no weak entity. MEDIA was the candidate: a photo or video means nothing without the post it belongs to, so MEDIA could have been weak, owned by POST.

In the tables

MEDIA(**id**, *post_id*, file_url, media_type)

media

id	post_id	file_url	media_type

No weak entity in the end: media keeps its own key, with a mandatory post_id

```
create table public.media (
  id          uuid              not null default gen_random_uuid(),
  post_id     uuid              not null,
  file_url    text              not null,
  media_type  public.media_type not null,     -- 'image' or 'video'
  constraint media_pkey primary key (id),
  constraint media_post_id_fkey foreign key (post_id)
    references posts (id) on delete cascade
);

create index idx_media_post_id on public.media using btree (post_id);
```

Why it matters

We gave MEDIA its own surrogate key, so one file can be found or removed by a single column. The owner rule is still kept: post_id is NOT NULL, and ON DELETE CASCADE deletes a post's media rows with the post (BR17). COMMENTS works the same way with its post (BR18).

TABLE MAPPING · STEP 3 OF 10

Composite and derived attributes → simple columns, or none

Rule

A composite attribute is replaced by its parts, each one a column of its own. A derived attribute gets no column: it is calculated when it is read.

In our EER

POST has two composite attributes, Edited (EditedAt, EditedBy) and Hidden (isHidden, HiddenAt, HiddenBy, HiddenReason), and three derived ones drawn with dashed ovals: ReactionCount, CommentCount and ReportCount. USER's Role (isCMUP, isMod) is composite too.

In the tables

POSTS(**id**, *user_id*, title, content, is_important, created_at, updated_at, edited_at, edited_by, is_hidden, hidden_at, hidden_by, hidden_reason)



Edited and Hidden flattened into one column each; the counts are derived and not stored

```
create table public.posts (
  id              uuid        not null default gen_random_uuid(),
  user_id         uuid        not null,
  title           text        not null default 'Untitle Post',
  content         text        not null,
  is_important    boolean     not null default false,
  created_at      timestamptz not null default now(),
  updated_at      timestamptz not null default now(),
  edited_at       timestamptz null,          -- Edited
  edited_by       uuid        null,
  is_hidden       boolean     not null default false,   -- Hidden
  hidden_at       timestamptz null,
  hidden_by       uuid        null,
  hidden_reason   text        null,
  constraint posts_pkey primary key (id),
  constraint posts_user_id_fkey foreign key (user_id)
    references users (id) on update cascade on delete cascade
);
```

Why it matters

Each part can now be read and filtered on its own; for example, the API finds the hidden posts with where is_hidden = true. The counts are not stored (BR42): the view v_post_all_data counts post_reactions, comments and reports when the feed is read, and a category's FollowerCount is counted from

user_interests. A stored count would need a second write on every reaction and could drift from the rows it summarises.

TABLE MAPPING · STEP 4 OF 10

Binary 1:N relationship → FK on the N side

Rule

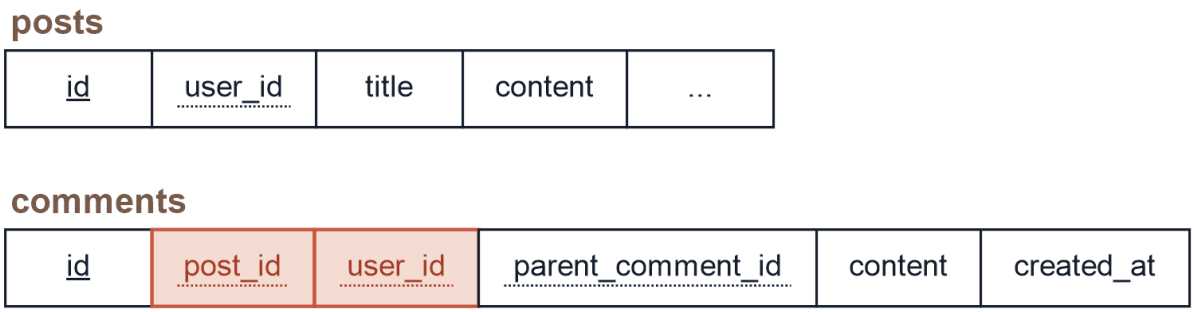
The primary key of the 1 side goes into the N side as a foreign key. If every row on the N side must take part (total participation), the FK is NOT NULL.

In our EER

POST 1:N COMMENT (Has) and USER 1:N COMMENT (Comments). Every comment belongs to exactly one post and is written by exactly one user.

In the tables

COMMENTS(**id**, *post_id*, *user_id*, *parent_comment_id*, content, created_at)



The foreign key sits on the many side and is declared not null

```
create table public.comments (
  id                uuid        not null default gen_random_uuid(),
  post_id           uuid        not null,      -- Has
  user_id           uuid        not null,      -- Comments
  parent_comment_id  uuid       null,          -- Replies to (step 6)
  content           text        not null,
  created_at        timestamptz not null default now(),
  constraint comments_pkey primary key (id),
  constraint comments_post_id_fkey foreign key (post_id)
    references posts (id) on delete cascade,
  constraint comments_user_id_fkey foreign key (user_id)
    references users (id) on delete cascade,
  constraint comments_parent_comment_id_fkey foreign key (parent_comment_id)
    references comments (id) on delete cascade
);
```

Why it matters

Our most common step, used 5 times: Writes (posts.user_id), Has (comments.post_id), Comments (comments.user_id), Contains (media.post_id) and Records (moderation_log.actor_id). All five N sides

are total, so every FK is NOT NULL. ON DELETE CASCADE removes a post's comments with the post (BR18).

TABLE MAPPING · STEP 5 OF 10

Partial participation → nullable FK

Rule

When the N side of a 1:N relationship may take no part (a zero in the cardinality), its foreign key allows NULL.

In our EER

MODERATOR 1:N REPORT (Review), with zero-or-one on the moderator side: a report that is still Open has not been reviewed by anyone.

In the tables

REPORTS(**id**, *post_id*, *user_id*, reason, status, reviewed_at, *reviewed_by*, created_at)

reports

id	post_id	user_id	reason	status	reviewed_at	reviewed_by	created_at

Nullable while the report is Open, filled in once a moderator reviews it

```
create table public.reports (
  id           uuid                   not null default gen_random_uuid(),
  post_id      uuid                   not null,
  user_id      uuid                   not null,
  reason       text                   not null,
  status       public.report_status   not null default 'Open',
  reviewed_at  timestamptz            null,
  created_at   timestamptz            not null default now(),
  reviewed_by  uuid                   null,
  constraint reports_pkey primary key (id),
  constraint reports_post_id_fkey foreign key (post_id)
    references posts (id) on delete cascade,
  constraint reports_user_id_fkey foreign key (user_id)
    references users (id) on update cascade on delete cascade,
  constraint reports_reviewed_by_fkey foreign key (reviewed_by)
    references users (id) on delete set null
);
```

Why it matters

reviewed_by stays NULL while a report is Open and is filled in when a moderator reviews or dismisses it (BR21). ON DELETE SET NULL keeps the report if the moderator's account is removed. The trigger trg_check_reviewer_is_moderator rejects a reviewer who is not a moderator (BR37). Hides works the same way: posts.hidden_by is NULL until a moderator hides the post.

TABLE MAPPING · STEP 6 OF 10

Recursive relationship → FK to the same table

Rule

A relationship between an entity and itself becomes a foreign key that points back at the primary key of the same table.

In our EER

COMMENT 1:N COMMENT (Replies to). A reply answers exactly one parent comment; a top-level comment has no parent, so the parent side is zero-or-one.

In the tables

COMMENTS(**id**, *post_id*, *user_id*, *parent_comment_id*, content, created_at)



A foreign key back into the same table; null marks a top-level comment

```
-- the column and key behind "Replies to"
parent_comment_id uuid null,
constraint comments_parent_comment_id_fkey foreign key (parent_comment_id)
  references comments (id) on delete cascade

create index idx_comments_parent_comment_id
  on public.comments using btree (parent_comment_id);

-- reading a whole thread: a comment and every reply below it
with recursive thread as (
  select id from comments where id = :comment_id
  union all
  select c.id from comments c join thread t on c.parent_comment_id = t.id
)
select id from thread;
```

Why it matters

NULL means a top-level comment and a value means a reply (BR19), so one table holds threads of any depth. ON DELETE CASCADE removes the replies with their parent. The foreign key cannot check that the parent is on the same post, so the API checks it before saving a reply (BR4).

TABLE MAPPING · STEP 7 OF 10

Binary M:N relationship → bridge table

Rule

A new table holds the keys of both sides as foreign keys; together they identify one row. The relationship's own attributes move into it.

In our EER

USER M:N CATEGORY (UserInterest: the categories a user follows) and POST M:N CATEGORY (PostCategory: the categories of a post).

In the tables

USER_INTERESTS(**id**, *user_id*, *category_id*, created_at) — UNIQUE(user_id, category_id)
POST_CATEGORIES(**id**, *post_id*, *category_id*, created_at) — UNIQUE(post_id, category_id)



A bridge table per M:N relationship, with UNIQUE on the pair of foreign keys

```
create table public.user_interests (
  id           uuid        not null default gen_random_uuid(),
  user_id      uuid        not null,
  category_id  uuid        not null,
  created_at   timestamptz not null default now(),
  constraint user_interests_pkey primary key (id),
  constraint uq_user_interest unique (user_id, category_id),
  constraint user_interests_category_id_fkey foreign key (category_id)
    references categories (id) on delete cascade,
  constraint user_interests_user_id_fkey foreign key (user_id)
    references users (id) on update cascade on delete cascade
);
```

Why it matters

Each bridge has its own surrogate id, but the pair of foreign keys is UNIQUE, so a user follows a category only once. The delete rules differ on purpose: deleting a category unfollows it for everyone (CASCADE), while post_categories.category_id uses ON DELETE RESTRICT, so a category that posts still use cannot be deleted (BR28). A constraint trigger makes every post keep at least one category (BR16).

TABLE MAPPING · STEP 8 OF 10

Associative entity → its own table

Rule

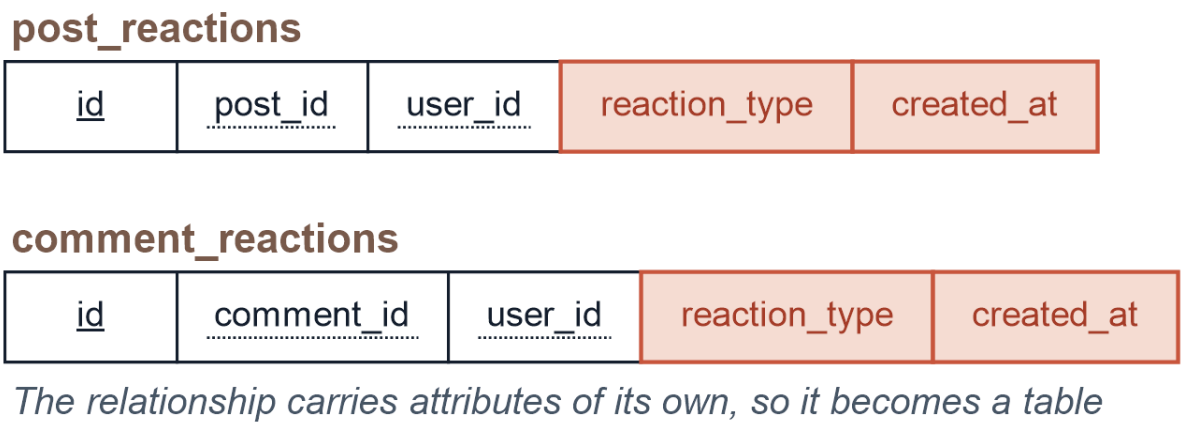
An M:N relationship that has its own attributes is drawn as an associative entity. It becomes a table with a foreign key to each side plus its own attributes.

In our EER

USER M:N POST appears three times, each with its own meaning and data: Post Reaction (ReactionType), Report (Reason, Status, ReviewedAt) and Notification (Message, IsRead). USER M:N COMMENT is Comment Reaction.

In the tables

POST_REACTIONS(**id**, *post_id*, *user_id*, reaction_type, created_at) — UNIQUE(post_id, user_id)



The relationship carries attributes of its own, so it becomes a table

```
create table public.post_reactions (
  id             uuid                not null default gen_random_uuid(),
  post_id        uuid                not null,
  user_id        uuid                not null,
  reaction_type  public.reaction_type not null,
  created_at     timestamptz         not null default now(),
  constraint post_reactions_pkey primary key (id),
  constraint uq_post_reaction unique (post_id, user_id),
  constraint post_reactions_post_id_fkey foreign key (post_id)
    references posts (id) on delete cascade,
  constraint post_reactions_user_id_fkey foreign key (user_id)
    references users (id) on delete cascade
);
```

Why it matters

reaction_type depends on the user and the post together, so it lives in the bridge (2NF). UNIQUE (post_id, user_id) allows one reaction per user per post (BR25); changing it updates the same row. REPORTS and NOTIFICATIONS have no such UNIQUE on purpose: a user may report a post again once the first report is closed (BR35), and may get several notifications about one post.

TABLE MAPPING · STEP 9 OF 10

Specialization → flag columns in the supertype table

Rule

When the subtypes have no attributes of their own, the supertype table can hold them all: each subtype becomes a flag column, and the rules of the specialization become constraints.

In our EER

USER is CMU Personnel and/or a Moderator: overlapping (o) and partial (single line). POST is either an Announcement or a Normal Post, decided by isImportant: disjoint (d) and total (double line).

In the tables

USERS(**id**, cmu_account, display_name, email, is_cmu_p, is_mod, created_at)
POSTS(**id**, *user_id*, title, content, is_important, …)

users

id	cmu_account	display_name	email	is_cmu_p	is_mod	created_at

posts

id	user_id	title	content	is_important	...

Flags in the supertype table instead of subtype tables

```
-- users: one flag per subtype (overlapping, partial)
is_cmu_p  boolean not null default false,   -- CMU Personnel
is_mod    boolean not null default false,   -- Moderator
constraint chk_mod_requires_cmup check (is_mod = false or is_cmu_p = true)

-- posts: one flag for a disjoint, total split
is_important  boolean not null default false  -- true = Announcement, false =
Normal Post
```

Why it matters

A table per subtype would hold nothing but a key. Two separate booleans let a user be either, both or neither, which is overlapping and partial (BR55); the CHECK makes every Moderator CMU Personnel (BR56). A single NOT NULL boolean is always exactly true or false, so a post is always one subtype and never both (BR57). Only a moderator may set is_important; the API checks this (BR13).

TABLE MAPPING · STEP 10 OF 10

Two roles on the same entity → two FKs

Rule

When an entity takes part in two relationships with the same entity, each relationship becomes its own foreign key, named after its role.

In our EER

USER takes part in REPORT twice: once as the reporter who files it, and once, as a Moderator, as the reviewer who decides it (Review).

In the tables

REPORTS(**id**, *post_id*, *user_id*, reason, status, reviewed_at, *reviewed_by*, created_at)

users

id	cmu_account	display_name	email	is_cmu_p	is_mod	created_at

reports

id	post_id	user_id	reason	status	reviewed_at	reviewed_by	created_at

Two foreign keys to users in two different roles: the reporter and the moderator

```
-- both foreign keys point at users
constraint reports_user_id_fkey foreign key (user_id)          -- reporter
  references users (id) on update cascade on delete cascade,
constraint reports_reviewed_by_fkey foreign key (reviewed_by)  -- reviewer
  references users (id) on delete set null

-- each role is joined to users separately
select r.reason, r.status,
       reporter.display_name as reported_by,
       reviewer.display_name as reviewed_by
from reports r
join users reporter      on reporter.id = r.user_id
left join users reviewer on reviewer.id = r.reviewed_by;
```

Why it matters

The two roles have different delete rules: deleting a reporter deletes their reports, but deleting a reviewer keeps the report and only clears reviewed_by. POSTS has the same pattern: user_id is the author, while edited_by and hidden_by name the moderator who edited or hid the post. Those two have no foreign key and are checked by the API (dashed lines in the mapping diagram), and moderation_log.post_id has none so a log entry outlives its post (BR22).

Normalization

The feed was first held in one relation that carried everything a post shows. Each step below removes one kind of dependency from it, and the twelve relations that result are the ones the database implements.

1. Unnormalised relation to 1NF (atomic values)

POST_FEED(**post_id**, title, media_url, category, user_id, display_name, is_mod, …)

post_id	title	media_url	category	user_id	display_name	is_mod
P1	Library close time	lib1.jpg, lib2.jpg	Warning, News	U1	WhongNaiMor Admin	true
P2	Advance Programming	(none)	Question	U2	Nat Wong	false
P3	IShowSpeed at CMU	sp1.jpg, sp2.jpg	Event, News	U1	WhongNaiMor Admin	true

media_url and category hold lists, so their values are not atomic and the relation is not in 1NF. Each repeating group becomes a row of its own in a relation of its own.

Repeating group	**Extracted into**	**BR**
The media files of a post	MEDIA(**media_id**, *post_id*, file_url, media_type)	BR5, BR47
The categories of a post	POST_CATEGORY(*post_id*, *category_id*, category_name, priority, created_at)	BR6, BR51

Every value is atomic now, but POST_CATEGORY has a composite key, which opens the way to a partial dependency.

2. Functional dependencies

The dependencies below are the ones that hold in the finished schema. The two that steps 3 and 4 remove are shown with those steps.

Relation	Functional dependencies
USERS	id → cmu_account, display_name, email, is_cmu_p, is_mod, created_at email → id (unique key, BR23)
CATEGORIES	id → category_name, description, priority category_name → id (unique key, BR27)
POSTS	id → user_id, title, content, is_important, created_at, updated_at, edited_at, edited_by, is_hidden, hidden_at, hidden_by, hidden_reason
COMMENTS	id → post_id, user_id, parent_comment_id, content, created_at
MEDIA	id → post_id, file_url, media_type
POST_CATEGORIES	id → post_id, category_id, created_at {post_id, category_id} → id, created_at (alternate key)
USER_INTERESTS	id → user_id, category_id, created_at {user_id, category_id} → id, created_at (alternate key)
POST_REACTIONS	id → post_id, user_id, reaction_type, created_at {post_id, user_id} → id, reaction_type (alternate key, BR25)
COMMENT_REACTIONS	id → comment_id, user_id, reaction_type, created_at {comment_id, user_id} → id, reaction_type (alternate key, BR25)
REPORTS	id → post_id, user_id, reason, status, reviewed_at, reviewed_by, created_at
NOTIFICATIONS	id → user_id, post_id, message, is_read, created_at
MODERATION_LOG	id → actor_id, action, post_id, post_title, target_id, detail, created_at

Every relation is determined by its primary key. The pairs listed for the four bridge relations are alternate candidate keys and are declared UNIQUE, because the surrogate key id does not enforce the pairing by itself.

3. From 1NF to 2NF (eliminating partial dependencies)

A relation is in 2NF when no non-key attribute depends on only part of the key. POST_CATEGORY, whose candidate key is {post_id, category_id}, breaks that rule:

{post_id, category_id} → created_at	full dependency, correct
category_id → category_name, priority	partial dependency, must be removed

A category name held on every row that uses it would have to be rewritten on all of those rows whenever the category is renamed. The category attributes were therefore moved into a relation of their own, together with description, which the Category entity carries in the EER model, leaving the bridge with the two foreign keys and created_at.

CATEGORIES(**id**, category_name, description, priority)
POST_CATEGORIES(**id**, *post_id*, *category_id*, created_at) — UNIQUE(post_id, category_id)

The same step applies to the other three bridge relations: user_interests, post_reactions and comment_reactions keep only their two foreign keys and the attributes that depend on both of them, namely reaction_type and created_at. Every relation is now in 2NF.

4. From 2NF to 3NF (eliminating transitive dependencies)

A relation is in 3NF when no non-key attribute determines another. POSTS breaks that rule:

post_id → user_id key dependency, correct
user_id → display_name, is_mod transitive dependency, must be removed

The author's display name repeated on every post they wrote would have to be rewritten on all of them whenever the name changes. The author attributes were moved into USERS, together with the remaining attributes of the User entity, and posts keeps only the foreign key.

USERS(**id**, cmu_account, display_name, email, is_cmu_p, is_mod, created_at)
POSTS(**id**, *user_id*, title, content, …)

The same transitive dependency was removed from comments, reports and moderation_log, which hold a foreign key to the user and none of the user's attributes.

Every determinant that remains — each id, email, category_name, and the natural key of each bridge relation — is a candidate key, so the schema is in Boyce–Codd normal form as well.

5. Three results that look unnormalised but are not

Where	Looks like	Why it is correct
moderation_log.post_title	A copy of posts.title, so post_id → post_title would be transitive	It records the title as it stood at the time of the action, so two entries about one post may differ. The entry must also outlive the post it describes, which a foreign key would prevent (BR22)
notifications.message	Contains an author name, a category name and a post title	The wording is fixed when the event happens and is never rewritten; rewriting it would misstate what the recipient was told (BR53)
users.is_cmu_p, users.is_mod	BR56 requires every moderator to be CMU personnel, which looks like is_mod → is_cmu_p	Not a dependency to remove: is_mod = false occurs with is_cmu_p both true and false, and the rule is enforced by the CHECK constraint chk_mod_requires_cmup

Derived values go the other way. The reaction, comment and report counts of a post and the follower count of a category are not stored (BR42): storing them would mean writing two

relations per reaction, and a failure between the two would leave the stored count disagreeing with the rows it summarises. They are computed on read by the view v_post_all_data.

6. Final normalized schema

Twelve relations, all in 3NF and BCNF. Notation: **primary key** in bold, *foreign key* in italic.

Core entities

USERS(**id**, cmu_account, display_name, email, is_cmu_p, is_mod, created_at) — UNIQUE(email)
CATEGORIES(**id**, category_name, description, priority) — UNIQUE(category_name)
POSTS(**id**, *user_id*, title, content, is_important, created_at, updated_at, edited_at, *edited_by*, is_hidden, hidden_at, *hidden_by*, hidden_reason)
COMMENTS(**id**, *post_id*, *user_id*, *parent_comment_id*, content, created_at)
MEDIA(**id**, *post_id*, file_url, media_type)

Bridge relations (M:N)

POST_CATEGORIES(**id**, *post_id*, *category_id*, created_at) — UNIQUE(post_id, category_id)
USER_INTERESTS(**id**, *user_id*, *category_id*, created_at) — UNIQUE(user_id, category_id)
POST_REACTIONS(**id**, *post_id*, *user_id*, reaction_type, created_at) — UNIQUE(post_id, user_id)
COMMENT_REACTIONS(**id**, *comment_id*, *user_id*, reaction_type, created_at) — UNIQUE(comment_id, user_id)

Moderation and notification

REPORTS(**id**, *post_id*, *user_id*, reason, status, reviewed_at, *reviewed_by*, created_at)
NOTIFICATIONS(**id**, *user_id*, *post_id*, message, is_read, created_at)
MODERATION_LOG(**id**, *actor_id*, action, post_id, post_title, target_id, detail, created_at)

DDL

Normal Table

```
-- ============================================================================
-- WhongNaiMor (วงในมอ) — Core Database Schema
-- Course: 953212 Database Systems / SE212 Database System and Design
-- Target: PostgreSQL 16 via Supabase
-- ============================================================================
-- Run this file first in the Supabase SQL Editor (or `supabase db push`
-- if you keep it as a migration file).
-- ============================================================================
```

```sql
-- ------------------------------------------------------------------------
-- 0. Extensions
-- ------------------------------------------------------------------------
create extension if not exists "pgcrypto";   -- gives us gen_random_uuid()

-- ------------------------------------------------------------------------
-- 1. Enum types
-- ------------------------------------------------------------------------
create type reaction_type as enum ('like', 'love', 'haha', 'sad', 'angry',
'care');
create type report_status as enum ('Open', 'Reviewed', 'Dismissed');
create type media_type    as enum ('image', 'video');

-- ========================================================================
-- 2. USERS
-- ========================================================================
-- NOTE (from proposal): for MVP there is no real CMU OAuth login, so
-- these are template/seed accounts. In production, `id` should be the
-- same uuid as `auth.users.id` (Supabase's built-in auth table), so
-- this table becomes a "profile" extension of auth.users.
create table users (
    id             uuid primary key default gen_random_uuid(),
    cmu_account    text not null unique,                 -- e.g.
"somchai_j@cmu.ac.th"
    display_name   text not null,
    email          text not null unique,
    is_cmu_p       boolean not null default false,        -- BR21: CMU
Personnel flag
    is_mod         boolean not null default false,        -- BR21: Moderator
flag
    created_at     timestamptz not null default now(),

    -- BR16: CMU account must end in @cmu.ac.th
    constraint chk_cmu_account_domain check (cmu_account ~*
'^[a-zA-Z0-9._%+-]+@cmu\.ac\.th$'),

    -- BR21: a user must be CMU Personnel *before* they can be a Moderator
    constraint chk_mod_requires_cmup check (is_mod = false or is_cmu_p = true)
);
```

```sql
comment on table users is 'Specialization: User -> CMU Personnel (is_cmu_p) ->
Moderator (is_mod), disjoint hierarchy per BR21/BR23.';

-- ============================================================================
-- 3. CATEGORIES
-- ============================================================================
-- Data is added by the dev/admin team only (not user-generated), e.g.
-- News, Event, Question, Lost & Found, Warning.
create table categories (
    id              uuid primary key default gen_random_uuid(),
    category_name   text not null unique,      -- BR18: unique category name
    description     text,
    priority        integer not null default 0,
    created_at      timestamptz not null default now()
);

-- ============================================================================
-- 4. POSTS
-- ============================================================================
create table posts (
    id              uuid primary key default gen_random_uuid(),
    user_id         uuid not null references users(id) on delete cascade,
    content         text not null,
    is_important    boolean not null default false,    -- specialization flag:
Announcement vs Normal Post
    created_at      timestamptz not null default now(),
    updated_at      timestamptz not null default now()
);

comment on table posts is 'Specialization: Post -> Announcement (is_important =
true) / Normal Post (is_important = false), disjoint, per the ERD "d"
specialization circle.';

create index idx_posts_user_id       on posts(user_id);
create index idx_posts_is_important  on posts(is_important) where is_important
= true;
create index idx_posts_created_at    on posts(created_at desc);

-- ============================================================================
-- 5. COMMENTS  (weak entity, owned by POST; recursive for "Replies to")
-- ============================================================================
create table comments (
```

```
    id                uuid primary key default gen_random_uuid(),
    post_id           uuid not null references posts(id) on delete cascade,
-- BR14
    user_id           uuid not null references users(id) on delete cascade,
    parent_comment_id  uuid references comments(id) on delete cascade,
-- BR4/BR15: null = top-level
    content           text not null,
    created_at        timestamptz not null default now()
);

comment on column comments.parent_comment_id is 'BR15: NULL for a top-level
comment; set for a reply. Enforces the recursive "Replies to" relationship.';

create index idx_comments_post_id           on comments(post_id);
create index idx_comments_parent_comment_id  on comments(parent_comment_id);



-- ============================================================================
-- 6. MEDIA (weak entity, owned by POST)
-- ============================================================================
create table media (
    id          uuid primary key default gen_random_uuid(),
    post_id     uuid not null references posts(id) on delete cascade,  -- BR13
    file_url    text not null,
    media_type  media_type not null,
    created_at  timestamptz not null default now()
);

create index idx_media_post_id on media(post_id);

-- ============================================================================
-- 7. POST REACTIONS  (bridge: User <-M:N-> Post)
-- ============================================================================
create table post_reactions (
    id             uuid primary key default gen_random_uuid(),
    post_id        uuid not null references posts(id) on delete cascade,
    user_id        uuid not null references users(id) on delete cascade,
    reaction_type  reaction_type not null,
    created_at     timestamptz not null default now(),

    -- BR17: a user can only react once to the same post
    constraint uq_post_reaction unique (post_id, user_id)
```

```sql
);

create index idx_post_reactions_post_id on post_reactions(post_id);

-- ============================================================================
-- 8. COMMENT REACTIONS  (bridge: User <-M:N-> Comment)
-- ============================================================================
create table comment_reactions (
    id             uuid primary key default gen_random_uuid(),
    comment_id     uuid not null references comments(id) on delete cascade,
    user_id        uuid not null references users(id) on delete cascade,
    reaction_type  reaction_type not null,
    created_at     timestamptz not null default now(),

    -- BR17: a user can only react once to the same comment
    constraint uq_comment_reaction unique (comment_id, user_id)
);

create index idx_comment_reactions_comment_id on comment_reactions(comment_id);

-- ============================================================================
-- 9. USER INTERESTS  (bridge: User <-M:N-> Category, "follow")
-- ============================================================================
create table user_interests (
    id           uuid primary key default gen_random_uuid(),
    user_id      uuid not null references users(id) on delete cascade,
    category_id  uuid not null references categories(id) on delete cascade,
    created_at   timestamptz not null default now(),

    constraint uq_user_interest unique (user_id, category_id)
);

-- ============================================================================
-- 10. POST CATEGORIES  (bridge: Post <-M:N-> Category, "tagging")
-- ============================================================================
create table post_categories (
    id           uuid primary key default gen_random_uuid(),
    post_id      uuid not null references posts(id) on delete cascade,
    category_id  uuid not null references categories(id) on delete restrict,
    created_at   timestamptz not null default now(),

    constraint uq_post_category unique (post_id, category_id)
```

```
);

create index idx_post_categories_post_id     on post_categories(post_id);
create index idx_post_categories_category_id on post_categories(category_id);

-- ============================================================================
-- 11. REPORTS  (bridge: User <-M:N-> Post, "flag content")
-- ============================================================================
create table reports (
    id            uuid primary key default gen_random_uuid(),
    post_id       uuid not null references posts(id) on delete cascade,
    user_id       uuid not null references users(id) on delete cascade,   --
reporter
    reason        text not null,
    status        report_status not null default 'Open',                  --
BR19
    reviewed_by   uuid references users(id),                               --
BR22: must be a moderator (checked by trigger below)
    reviewed_at   timestamptz,
    created_at    timestamptz not null default now(),

    -- a user reporting the same post twice is unusual but not forbidden
    -- by the business rules, so no uniqueness constraint here.
    constraint chk_reviewed_fields_together
        check ( (reviewed_by is null and reviewed_at is null) or (reviewed_by
is not null and reviewed_at is not null) )
);

create index idx_reports_post_id on reports(post_id);
create index idx_reports_status  on reports(status) where status = 'Open';

-- ============================================================================
-- 12. NOTIFICATIONS  (bridge: User <-M:N-> Post, "Announcement fan-out")
-- ============================================================================
create table notifications (
    id          uuid primary key default gen_random_uuid(),
    user_id     uuid not null references users(id) on delete cascade,   --
recipient
    post_id     uuid not null references posts(id) on delete cascade,   -- the
Announcement post
    message     text not null,
    is_read     boolean not null default false,
```

```
    created_at  timestamptz not null default now()
);

create index idx_notifications_user_id           on notifications(user_id);
create index idx_notifications_user_id_is_read    on notifications(user_id)
where is_read = false;
```

View Table

v_comments

```
create view public.v_comments
with
  (security_invoker = on) as
select
  c.id,
  c.post_id,
  c.parent_comment_id,
  u.display_name,
  case
    when u.is_mod
    and u.is_cmu_p then 'mod'::text
    when u.is_cmu_p then 'student'::text
    else null::text
  end as role,
  c.content,
  c.created_at
from
  comments c
  join users u on u.id = c.user_id;
```

v_notification

```
create view public.v_notification
with
  (security_invoker = on) as
select
  p.id,
  p.content,
  p.created_at,
  n.user_id as receiver,
```

```
  u.display_name as author_name,
  COALESCE(
    (
      select
        jsonb_agg(
          jsonb_build_object(
            'name',
            categories.category_name,
            'description',
            categories.description,
            'priority',
            categories.priority
          )
          order by
            categories.priority
        ) as jsonb_agg
      from
        post_categories
        join categories on categories.id = post_categories.category_id
      where
        post_categories.post_id = p.id
    ),
    '[]'::jsonb
  ) as categories,
  n.user_id as sent_to,
  n.message as notf_msg,
  n.is_read,
  n.created_at as notified_at
from
  posts p
  join users u on u.id = p.user_id
  join notifications n on n.post_id = p.id
where
  p.is_important = true
order by
  p.created_at desc;
```

v_open_reports

```
create view public.v_open_reports
with
  (security_invoker = on) as
select
```

```
  rp.id as report_id,
  rp.reason,
  rp.created_at as reported_at,
  p.id as post_id,
  p.content as post_content,
  reporter.display_name as reported_by
from
  reports rp
  join posts p on p.id = rp.post_id
  join users reporter on reporter.id = rp.user_id
where
  rp.status = 'Open'::report_status
order by
  rp.created_at;
```

v_post_all_data

```
create view public.v_post_all_data
with
  (security_invoker = on) as
select
  p.id,
  p.title,
  p.content,
  p.is_important,
  COALESCE(
    (
      select
        jsonb_agg(
          jsonb_build_object(
            'name',
            categories.category_name,
            'description',
            categories.description,
            'priority',
            categories.priority
          )
          order by
            categories.priority
        ) as jsonb_agg
      from
        post_categories
        join categories on categories.id = post_categories.category_id
```

```
      where
        post_categories.post_id = p.id
    ),
    '[]'::jsonb
  ) as categories,
  COALESCE(
    (
      select
        jsonb_agg(
          jsonb_build_object(
            'file_url',
            media.file_url,
            'media_type',
            media.media_type
          )
        ) as jsonb_agg
      from
        media
      where
        media.post_id = p.id
    ),
    '[]'::jsonb
  ) as medias,
  u.display_name as author_name,
  case
    when u.is_mod
    and u.is_cmu_p then 'mod'::text
    when u.is_cmu_p then 'student'::text
    else null::text
  end as role,
  COALESCE(r.reaction_count, 0::bigint) as reaction_count,
  COALESCE(cm.comment_count, 0::bigint) as comment_count,
  COALESCE(rp.report_count, 0::bigint) as report_count,
  COALESCE(r.reaction_count, 0::bigint) + COALESCE(cm.comment_count, 0::bigint)
as engagement_score,
  p.created_at
from
  posts p
  join users u on u.id = p.user_id
  left join (
    select
      post_reactions.post_id,
```

```
      count(*) as reaction_count
    from
      post_reactions
    group by
      post_reactions.post_id
  ) r on r.post_id = p.id
  left join (
    select
      comments.post_id,
      count(*) as comment_count
    from
      comments
    group by
      comments.post_id
  ) cm on cm.post_id = p.id
  left join (
    select
      reports.post_id,
      count(*) as report_count
    from
      reports
    group by
      reports.post_id
  ) rp on rp.post_id = p.id
order by
  (
    COALESCE(r.reaction_count, 0::bigint) + COALESCE(cm.comment_count,
0::bigint)
  ) desc;
```

v_question_posts

```
create view public.v_question_posts
with
  (security_invoker = on) as
select
  p.id,
  p.content,
  p.created_at,
  u.display_name as author_name,
  c.category_name
from
  posts p
```

```
  join users u on u.id = p.user_id
  join post_categories pc on pc.post_id = p.id
  join categories c on c.id = pc.category_id
where
  c.category_name ~~* 'question%'::text
order by
  p.created_at desc;
```

v_user_all_data

```
create view public.v_user_all_data
with
  (security_invoker = on) as
select
  id,
  cmu_account,
  display_name,
  email,
  is_cmu_p,
  is_mod,
  created_at,
  COALESCE(
    (
      select
        jsonb_agg(
          jsonb_build_object(
            'name',
            categories.category_name,
            'description',
            categories.description,
            'priority',
            categories.priority
          )
          order by
            categories.priority
        ) as jsonb_agg
      from
        user_interests
        join categories on categories.id = user_interests.category_id
      where
        user_interests.user_id = u.id
    ),
    '[]'::jsonb
```

```
  ) as u_interest_cs
from
  users u;
```

DML

Supabase SDK SQL equivalent code

The API talks to PostgreSQL through the Supabase SDK (supabase-js). Each SDK call below is turned into SQL by PostgREST, the Supabase Storage service or Supabase Auth. The table shows the SQL each call runs, what it does, and what the user sees on the website. Values written as :name are filled in by the API as bound parameters.

#	Supabase SDK	SQL equivalent code	What it does and how it affects the website
Sign-in, profile and followed categories			

Q1	`supabase.auth.getUser(access_token)`	`-- Supabase Auth checks the JWT signature, then`
		`SELECT id, email,`
		`raw_user_meta_data`
		`FROM auth.users`
		`WHERE id = :user_id_in_token;`
			Every request that needs sign-in (middleware.ts)
			Finds who is calling. If the token is missing or invalid the API answers 401, and the website signs the user out and shows the login dialog.

Q2	`supabase.auth.signInWithPassword({ email, password })`	`-- Supabase Auth`
		`SELECT id, encrypted_password`
		`FROM auth.users`
		`WHERE email = :email;`
		`-- password checked with bcrypt, then`
		`INSERT INTO auth.sessions`
		`(user_id, created_at)`
		`VALUES (:id, now());`
			Login dialog (frontend AuthContext.tsx, dbauth.ts)
			Signs the user in and gives the browser a token. The top bar then shows the user's name and the bell.

Q3	`from("v_user_all_data").select("*")`
	`.eq("id", uid).single()`
		`SELECT *`
		`FROM v_user_all_data`
		`WHERE id = :uid;`
		`-- .single(): exactly one row is expected`
			GET /user/current/fetch, GET /user/:uid/fetch, and every moderator check (users.ts)
			Loads the user's profile: name, email, isMod and isCMUP. Decides whether the Moderation tab and the "announcement" option appear, and fills the profile page.

Q4	`from("users").update({ display_name })`
	`.eq("id", userId)`
	`auth.admin.updateUserById(userId,`
	`{ user_metadata: { display_name } })`
		`UPDATE users`
		`SET display_name = :display_name`
		`WHERE id = :user_id;`
		
		`UPDATE auth.users`
		`SET raw_user_meta_data =`
		`raw_user_meta_data`
		`||`
		`jsonb_build_object('display_name', :display_name)`
		`WHERE id = :user_id;`
			PUT /user/me
			Renames the user from the profile page. The new name appears on all of their posts and comments.

#	Supabase SDK	SQL equivalent code	What it does and how it affects the website

Q5	`from("user_interests")`
	`  .select("category_id,`
	
	`category:categories(categ`
	`ory_name)")`
	`  .eq("user_id", userId)`
		`SELECT ui.category_id,`
		`c.category_name`
		`FROM user_interests ui`
		`LEFT JOIN categories c ON c.id =`
		`ui.category_id`
		`WHERE ui.user_id = :user_id;`
			GET /user/interests
			Lists the categories the user follows. These get a filled star in the sidebar and make up the "Following" feed.

Q6	`rpc("user_interests_op",`
	`  { uid, cate_id })`
		`SELECT user_interests_op(`
		`  uid     => :user_id,`
		`  cate_id => :category_ids   --`
		`uuid[]`
		`);`
		`-- replaces the user's rows in`
		`user_interests`
		`-- with exactly this list`
		`-- See user_interests_op`
		`function in RPC function`
		`section.`
			PUT /user/interests
			Saves follow / unfollow when the user clicks a star. From then on, new posts in those categories send them a notification.

Feed, search and post pages			

Q7	`from("v_post_all_data").se`
	`lect("*")`
	`  .order("created_at",`
	`desc).order("id", desc)`
	`  .in("id", only)`
	`  .eq("is_important",`
	`true)`
	`  .limit(n + 1)`
	`  .or("created_at.lt.<at>,`
	
	`and(created_at.eq.<at>,id.`
	`lt.<id>)")`
		`SELECT *`
		`FROM v_post_all_data`
		`WHERE id IN (:filtered_ids)`
		`-- category / following filter`
		`  AND is_important = true`
		`-- "important" filter only`
		`  AND (created_at < :at`
		`       OR (created_at = :at AND`
		`id < :id))   -- next page`
		`ORDER BY created_at DESC, id`
		`DESC`
		`LIMIT :page_size + 1;`
			GET /post/fetch (feed.ts)
			Loads the main feed, newest first. Scrolling down fetches the next page after the last post shown (infinite scroll). The extra row tells the site whether more pages exist.

Q8	`from("post_categories").se`
	`lect("post_id")`
	`  .in("category_id", ids)`
	`from("user_interests").sel`
	`ect("category_id")`
	`  .eq("user_id", viewerId)`
		`SELECT DISTINCT post_id`
		`FROM post_categories`
		`WHERE category_id IN`
		`(:category_ids);`
		
		`SELECT category_id`
		`FROM user_interests`
		`WHERE user_id = :viewer_id;`
			Feed filters (posts.ts filteredPostIds)
			Works out which posts belong to the categories picked in the sidebar, or to the categories the user follows ("Following" tab).

Q9	`from("posts")`
	`  .select("id, user_id,`
	`hidden_reason, hidden_at")`
	`  .eq("is_hidden", true)`
		`SELECT id, user_id,`
		`hidden_reason, hidden_at`
		`FROM posts`
		`WHERE is_hidden = true;`
			Every feed and post read (hidden.ts)
			Finds posts a moderator has hidden. They are removed from the feed for everyone except the author, who sees the post greyed out with the reason.

Q10	`from("posts").select("is_h`
	`idden")`
	`  .eq("id",`
	`postId).maybeSingle()`
		`SELECT is_hidden`
		`FROM posts`
		`WHERE id = :post_id;`
		`-- .maybeSingle(): zero or one`
		`row`
			Before reacting or commenting (hidden.ts)
			Blocks new reactions and comments on a hidden post; the site shows a message instead.

Q11	`from("posts")`
	`  .select("id, user_id,`
	`edited_at, edited_by")`
	`  .in("id", ids)`
		`SELECT id, user_id, edited_at,`
		`edited_by`
		`FROM posts`
		`WHERE id IN (:post_ids);`
			Every list of posts (posts.ts withOwnership)
			Marks which posts are the viewer's own, so the edit / delete menu appears on them, and shows the "edited" or "edited by a moderator" label.

#	Supabase SDK	SQL equivalent code	What it does and how it affects the website

Q12	`from("post_reactions")`
	`  .select("post_id, reaction_type")`
	`  .eq("user_id", userId).in("post_id", ids)`
	`(same for comment_reactions)`
		`SELECT post_id, reaction_type`
		`FROM post_reactions`
		`WHERE user_id = :viewer_id`
		`  AND post_id IN (:post_ids);`
			Feed and comments (reactions.ts myReactions)
			Highlights the reaction the viewer already gave on each post and comment.

Q13	`from("comment_reactions")`
	`  .select("comment_id")`
	`  .in("comment_id", ids)`
	`(counted in the API)`
		`SELECT comment_id, COUNT(*) AS reaction_count`
		`FROM comment_reactions`
		`WHERE comment_id IN (:comment_ids)`
		`GROUP BY comment_id;`
			Comments (reactions.ts reactionCounts)
			Shows the number of reactions under each comment.

Q14	`from("v_post_all_data").select("*")`
	`  .or("title.ilike.%q%, content.ilike.%q%, author_name.ilike.%q%")`
	`  .order("created_at", desc).limit(100)`
		`SELECT *`
		`FROM v_post_all_data`
		`WHERE title       ILIKE '%' || :q || '%'`
		`   OR content     ILIKE '%' || :q || '%'`
		`   OR author_name ILIKE '%' || :q || '%'`
		`ORDER BY created_at DESC`
		`LIMIT 100;`
			GET /post/search
			Search box in the top bar: finds posts whose title, text or author name contains the words, ignoring case.

Q15	`from("posts").select("id")`
	`  .eq("user_id", userId)`
	`from("v_post_all_data").select("*")`
	`  .in("id", ids).order("created_at", desc)`
		`SELECT *`
		`FROM v_post_all_data`
		`WHERE id IN (SELECT id FROM posts`
		`             WHERE user_id = :user_id)`
		`ORDER BY created_at DESC;`
			GET /post/mine
			"My posts" on the profile page, including the user's own hidden posts.

Q16	`from("v_post_all_data").select("*")`
	`  .eq("is_important", true)`
	`  .order("created_at", desc).limit(5)`
		`SELECT *`
		`FROM v_post_all_data`
		`WHERE is_important = true`
		`ORDER BY created_at DESC`
		`LIMIT 5;`
			GET /post/fetch/announcement
			The five newest announcements in the right-hand column.

Q17	`from("v_post_all_data").select("*")`
	`  .eq("id", pid).maybeSingle()`
		`SELECT *`
		`FROM v_post_all_data`
		`WHERE id = :post_id;`
			GET /post/:pid/fetch
			Opens one post from a link or a notification. A missing post, or a hidden one seen by someone other than its author or a moderator, shows "post not found".

Q18	`from("categories").select("*")`
	`.order("priority").order("category_name")`
	`from("user_interests").select("category_id")`
	`(followers counted in the API)`
		`SELECT c.*, COUNT(ui.user_id) AS follower_count`
		`FROM categories c`
		`LEFT JOIN user_interests ui ON ui.category_id = c.id`
		`GROUP BY c.id`
		`ORDER BY c.priority, c.category_name;`
			GET /misc/categories
			The category list in the sidebar and in the create-post dialog, most important first, with how many people follow each one.

Creating, editing and deleting posts			

Q19	`from("posts").insert({ user_id, title,`
	`  content, is_important`
	`}).select().single()`
		`INSERT INTO posts (user_id, title, content, is_important)`
		`VALUES (:user_id, :title, :content, :is_important)`
		`RETURNING *;`
			POST /post/create
			Saves a new post from the create-post dialog. is_important is only true when a moderator ticks "announcement". The post appears at the top of the feed.

#	Supabase SDK	SQL equivalent code	What it does and how it affects the website

Q20	`from("post_categories").insert(`
	`  category_id.map(cid =>`
	`  ({ post_id, category_id: cid })))`
		`INSERT INTO post_categories (post_id, category_id)`
		`VALUES (:post_id, :category_1),`
		`  (:post_id, :category_2),`
		`  ...;`
			POST /post/create
			Links the post to the categories chosen in the dialog; they show as tags on the post.

Q21	`storage.from("post_medias")`
	`  .upload(path, file, { upsert: true })`
	
	`storage.from("post_medias").getPublicUrl(path)`
	
	`from("media").insert(uploaded)`
		`-- Supabase Storage (file bytes go to object storage)`
		`INSERT INTO storage.objects (bucket_id, name, metadata)`
		`VALUES ('post_medias', :post_id || '/images/' || :md5 || '.jpg',`
		
		`jsonb_build_object('mimetype', :mime))`
		`ON CONFLICT (bucket_id, name) DO UPDATE`
		`  SET metadata = EXCLUDED.metadata;`
		
		`-- getPublicUrl only builds a URL: no query`
		
		`INSERT INTO media (post_id, file_url, media_type)`
		`VALUES (:post_id, :public_url, 'image');`
			POST /post/create (posts.ts uploadPostMedia)
			Stores the photos (up to 10) and video (1) attached to a post. They show under the post and open in the image viewer.

Q22	`from("user_interests")`
	`  .select("user_id, category_id")`
	`  .in("category_id", ids).neq("user_id", author)`
	
	`from("categories").select("id, category_name")`
	`  .in("id", ids)`
	
	`from("notifications").select("user_id")`
	`  .eq("post_id", id).in("user_id", ...)`
	`  .like("message", prefix + "%")`
	
	`from("notifications").insert(rows)`
		`SELECT user_id, category_id`
		`FROM user_interests`
		`WHERE category_id IN (:category_ids)`
		`  AND user_id <> :author_id;`
		
		`SELECT id, category_name`
		`FROM categories WHERE id IN (:category_ids);`
		
		`-- followers already told about this post`
		`SELECT user_id FROM notifications`
		`WHERE post_id = :post_id`
		`  AND user_id IN (:follower_ids)`
		`  AND message LIKE :prefix || '%';`
		
		`INSERT INTO notifications (user_id, post_id, is_read, message)`
		`VALUES (:follower_id, :post_id, false, :message);`
			After creating a post, or turning one into an announcement (followers.ts)
			Each follower of the post's categories gets one bell notification ("<name> posted in <category>: <title>" or "New announcement in <category>"). The author is not notified, and nobody is notified twice.

Q23	`rpc("react_to_post", { pid, uid,`
	`  rtype: reaction_type,`
	`  change_reaction: change })`
		`SELECT react_to_post(`
		`  pid  => :post_id,`
		`  uid  => :user_id,`
		`  rtype  => :reaction_type,`
		`  change_reaction => :change`
		`);`
		`-- adds, switches or removes the user's row in`
		`-- post_reactions and returns the new total`
		`-- See react_to_post function in RPC function section.`
			POST /post/:pid/react
			Clicking a reaction on a post. Clicking the same one again removes it; the count beside the button updates.

#	Supabase SDK	SQL equivalent code	What it does and how it affects the website

Q24	`from("posts")`
	`  .select("id, user_id, title, is_important")`
	`  .eq("id", pid).maybeSingle()`
		`SELECT id, user_id, title, is_important`
		`FROM posts`
		`WHERE id = :post_id;`
			Before editing or deleting a post (posts.ts postAccess)
			Checks that the post exists and that the caller is its author or a moderator; otherwise the site shows an error.

Q25	`from("posts").update({ title, content,`
	`  is_important, edited_at, edited_by })`
	`  .eq("id", pid)`
	`rpc("update_post_cate",`
	`  { pid, cate_name: category_names })`
		`UPDATE posts`
		`SET title = :title,`
		`  content = :content,`
		`  is_important = :is_important,  -- moderators only`
		`  edited_at = now(),`
		`  edited_by = :user_id`
		`WHERE id = :post_id;`
		
		`SELECT update_post_cate(`
		`  pid  => :post_id,`
		`  cate_name => :category_names`
		`-- text[]`
		`);`
		`-- replaces the post's rows in post_categories`
		`-- See update_post_cate function in RPC function section.`
			PUT /post/:pid/edit
			Saves the edit dialog. The post shows "edited" (or "edited by a moderator") and its new category tags.

Q26	`from("post_categories").select("category_id")`
	`  .eq("post_id", pid)`
		`SELECT category_id`
		`FROM post_categories`
		`WHERE post_id = :post_id;`
			PUT /post/:pid/edit, when a moderator turns a post into an announcement
			Finds the post's categories so their followers can be told about the new announcement.

Q27	`from("media").select("file_url").eq("post_id", pid)`
	
	`from("posts").delete().eq("id", pid).select("id")`
	
	`-- on foreign-key error 23503:`
	`(However, all the fk are set as cascaded on delete, so this is unlikely to be used)`
	`from("comments").select("id").eq("post_id", pid)`
	
	`from("comment_reactions").delete()`
	`  .in("comment_id", ids)`
	
	`from(table).delete().eq("post_id", pid)`
	`-- end foreign-key error`
	
	`storage.from("post_medias").remove(files)`
		`SELECT file_url FROM media WHERE post_id = :post_id;`
		
		`DELETE FROM posts WHERE id = :post_id RETURNING id;`
		
		`-- only if that fails with 23503 (rows still point at it):`
		`DELETE FROM comment_reactions`
		`WHERE comment_id IN (SELECT id FROM comments`
		`  WHERE post_id = :post_id);`
		`DELETE FROM comments`
		`WHERE post_id = :post_id;`
		`DELETE FROM post_reactions`
		`WHERE post_id = :post_id;`
		`DELETE FROM post_categories`
		`WHERE post_id = :post_id;`
		`DELETE FROM media`
		`WHERE post_id = :post_id;`
		`DELETE FROM reports`
		`WHERE post_id = :post_id;`
		`DELETE FROM notifications`
		`WHERE post_id = :post_id;`
		`DELETE FROM posts WHERE id = :post_id RETURNING id;`
		
		`-- Supabase Storage`
		`DELETE FROM storage.objects`
		`WHERE bucket_id = 'post_medias'`
		`AND name IN (:file_paths);`
			DELETE /post/:pid/delete (posts.ts deletePostCascade)
			Deletes a post with everything attached to it, including its photos and videos. It disappears from every feed. The website waits a few seconds before sending this request, so the user can still press Undo.

#	Supabase SDK	SQL equivalent code	What it does and how it affects the website
Comments			

Q28	`from("v_comments").select("*")`
	`  .in("post_id", pids)`
	
	`from("comments").select("id, user_id")`
	`  .in("id", ids)`
		`SELECT *`
		`FROM v_comments`
		`WHERE post_id IN (:post_ids);`
		
		`SELECT id, user_id`
		`FROM comments`
		`WHERE id IN (:comment_ids);`
			GET /comment/fetch/:pid, GET /comment/fetch-many
			Loads the comment threads under the posts on screen, and marks the viewer's own comments so they can be edited or deleted.

Q29	`from("posts").select("user_id, title, is_hidden")`
	`  .eq("id", pid).maybeSingle()`
	
	`from("comments").select("user_id, post_id")`
	`  .eq("id", cid).maybeSingle()`
	
	`from("comments").insert(comment)`
		`SELECT user_id, title, is_hidden`
		`FROM posts WHERE id = :post_id;`
		
		`-- replies only: the parent must be on the same post`
		`SELECT user_id, post_id`
		`FROM comments WHERE id = :parent_id;`
		
		`INSERT INTO comments (post_id, user_id, content, parent_comment_id)`
		`VALUES (:post_id, :user_id, :content, :parent_id);`
			POST /comment/:pid, POST /comment/:pid/replyto/:cid
			Adds a comment or a reply. It appears in the thread at once for everyone viewing the post.

Q30	`from("notifications").insert(rows)`	`INSERT INTO notifications (user_id, post_id, message)`
		`VALUES (:post_author, :post_id, :message),`
		`  (:parent_author, :post_id, :message);`
			After a comment, hide, unhide or report decision (notify.ts)
			Puts a message in the bell of the people affected, e.g. "<name> commented on your post" or "<name> replied to your comment".

Q31	`from("comments").select("post_id")`
	`  .eq("id", cid).maybeSingle()`
	
	`rpc("react_to_comment", { cid, uid,`
	`  rtype, change_reaction })`
		`SELECT post_id FROM comments`
		`WHERE id = :comment_id;`
		
		`SELECT react_to_comment(`
		`  cid  => :comment_id,`
		`  uid  => :user_id,`
		`  rtype  => :reaction_type,`
		`  change_reaction => :change`
		`);`
		
		`-- See react_to_comment function in RPC function section.`
			POST /comment/:cid/react
			Clicking a reaction on a comment; the count under the comment updates.

Q32	`from("comments").select("id, user_id, post_id")`
	`  .eq("id", cid).maybeSingle()`
	
	`from("comments").update({ content })`
	`  .eq("id", cid)`
		`SELECT id, user_id, post_id`
		`FROM comments WHERE id = :comment_id;`
		
		`UPDATE comments`
		`SET content = :content`
		`WHERE id = :comment_id;`
			PUT /comment/:cid
			Saves an edited comment. Only its author may do this.

#	Supabase SDK	SQL equivalent code	What it does and how it affects the website

Q33	`from("comments").select("id")`
	`  .in("parent_comment_id", frontier)  -- repeated`
	`from("comment_reactions").delete()`
	`  .in("comment_id", ids)`
	`from("comments").delete().eq("id", id)`
		`WITH RECURSIVE thread AS (`
		`  SELECT id FROM comments WHERE id = :comment_id`
		`  UNION ALL`
		`  SELECT c.id FROM comments c`
		`  JOIN thread t ON c.parent_comment_id = t.id`
		`)`
		`SELECT id FROM thread;`
		
		`DELETE FROM comment_reactions`
		`WHERE comment_id IN (:thread_ids);`
		
		`-- deepest replies first`
		`DELETE FROM comments WHERE id = :id;`
			DELETE /comment/:cid
			Deletes a comment together with all replies under it. The author or a moderator may do this; a moderator's deletion is written to the moderation log.

Notifications			

Q34	`from("notifications")`
	`  .select("id, post_id, message, is_read, created_at")`
	`  .eq("user_id", userId)`
	`  .order("created_at", desc)`
		`SELECT id, post_id, message, is_read, created_at`
		`FROM notifications`
		`WHERE user_id = :user_id`
		`ORDER BY created_at DESC;`
			GET /notification/fetch
			Fills the bell menu, newest first. Unread rows set the red badge number.

Q35	`from("notifications").update({ is_read: true })`
	`  .eq("id", nid).eq("user_id", userId)`
		`UPDATE notifications`
		`SET is_read = true`
		`WHERE id = :notification_id`
		`  AND user_id = :user_id;`
			POST /notification/:nid/read
			Clicking one notification marks it read and opens its post; the badge goes down by one.

Q36	`from("notifications").update({ is_read: true })`
	`  .eq("user_id", userId).eq("is_read", false)`
	`  .select("id")`
		`UPDATE notifications`
		`SET is_read = true`
		`WHERE user_id = :user_id`
		`  AND is_read = false`
		`RETURNING id;`
			POST /notification/read-all
			"Mark all as read" in the bell menu; the badge disappears.

Reports			
Q37	`from("reports")` `  .select("post_id, user_id, status, created_at")`	`SELECT post_id, user_id, status, created_at` `FROM reports;`	**GET /report/status** Shows the report status badge on posts, and stops a user reporting the same post twice.

Q38	`from("posts").select("id")`
	`.eq("id", post_id)`
	
	`from("reports").select("id")`
	`  .eq("post_id", post_id).eq("user_id", userId)`
	`  .eq("status", "Open").limit(1)`
	
	`from("reports").insert(reportData)`
	`  .select().single()`
		`SELECT id FROM posts WHERE id = :post_id;`
		
		`SELECT id FROM reports`
		`WHERE post_id = :post_id`
		`  AND user_id = :user_id`
		`  AND status = 'Open'`
		`LIMIT 1;`
		
		`INSERT INTO reports (post_id, user_id, reason, status)`
		`VALUES (:post_id, :user_id, :reason, 'Open')`
		`RETURNING *;`
			POST /report/createticket
			Sends the report dialog. The post goes into the moderators' queue; a second report by the same user is refused.

#	Supabase SDK	SQL equivalent code	What it does and how it affects the website
Q39	`from("reports").select("*, post:posts(id, title, content), reporter:users!reports_user_id_fkey(display_name)") .order("created_at", asc)`	`SELECT r.*, p.id, p.title, p.content, u.display_name AS reporter FROM reports r LEFT JOIN posts p ON p.id = r.post_id LEFT JOIN users u ON u.id = r.user_id ORDER BY r.created_at ASC;`	**GET /report/fetch (moderators)** The report queue on the Moderation page, oldest first, with the post and who reported it.
Q40	`from("reports") .select("id, post_id, reason, status, created_at, reviewed_at, post:posts(title)") .eq("user_id", userId) .order("created_at", desc)`	`SELECT r.id, r.post_id, r.reason, r.status, r.created_at, r.reviewed_at, p.title FROM reports r LEFT JOIN posts p ON p.id = r.post_id WHERE r.user_id = :user_id ORDER BY r.created_at DESC;`	**GET /report/mine** "My reports" on the profile page, with each report's status. A deleted post shows as "post deleted".
Q41	`from("reports").update({ status, reviewed_by, reviewed_at }) .eq("post_id", pid) .select("user_id, post:posts(title)")`	`UPDATE reports SET status = :status, -- 'Open' | 'Reviewed' | 'Dismissed' reviewed_by = :moderator_id, reviewed_at = now() WHERE post_id = :post_id RETURNING user_id, (SELECT title FROM posts WHERE id = reports.post_id) AS title;`	**PUT /report/post/:pid/status (moderators)** A moderator's decision on all reports of a post (sent after the same short Undo delay). The reporters get a notification, and the post leaves the open queue.
Q42	`from("reports").update({ status, reviewed_by, reviewed_at }) .eq("id", rid).select()`	`UPDATE reports SET status = :status, reviewed_by = :moderator_id, reviewed_at = now() WHERE id = :report_id RETURNING *;`	**PUT /report/review/:rid (moderators)** Changes the status of a single report.
Moderation			

Q43	`from("posts").update({ is_hidden: true, hidden_at, hidden_by, hidden_reason }) .eq("id", pid).select("id, user_id, title")`
	
	`from("reports").update({ status: "Reviewed", reviewed_by, reviewed_at }) .eq("post_id", pid).eq("status", "Open") .select("user_id")`
		`UPDATE posts SET is_hidden = true, hidden_at = now(), hidden_by = :moderator_id, hidden_reason = :reason WHERE id = :post_id RETURNING id, user_id, title;`
		
		`UPDATE reports SET status = 'Reviewed', reviewed_by = :moderator_id, reviewed_at = now() WHERE post_id = :post_id AND status = 'Open' RETURNING user_id;`
			PUT /post/:pid/hide (moderators) Hides a post from the feed and closes its open reports. The author and the reporters are notified.

Q44	`from("posts").select("id, title, hidden_reason, hidden_at, author:users!posts_user_id_fkey(display_name)") .eq("is_hidden", true) .order("hidden_at", desc)`	`SELECT p.id, p.title, p.hidden_reason, p.hidden_at, u.display_name AS author FROM posts p LEFT JOIN users u ON u.id = p.user_id WHERE p.is_hidden = true ORDER BY p.hidden_at DESC;`	**GET /post/hidden (moderators)** The "Hidden posts" list on the Moderation page.

#	Supabase SDK	SQL equivalent code	What it does and how it affects the website
Q45	`from("posts").update({ is_hidden: false, hidden_at: null, hidden_by: null, hidden_reason: null }) .eq("id", pid).select("id, user_id, title")`	`UPDATE posts SET is_hidden = false, hidden_at = NULL, hidden_by = NULL, hidden_reason = NULL WHERE id = :post_id RETURNING id, user_id, title;`	**PUT /post/:pid/unhide (moderators)** Shows a hidden post again; its author is notified that it is back.
Q46	`from("moderation_log").insert({ actor_id, action, post_id, post_title, target_id, detail })`	`INSERT INTO moderation_log (actor_id, action, post_id, post_title, target_id, detail) VALUES (:moderator_id, :action, :post_id, :post_title, :target_id, :detail);`	**After every moderator action (modlog.ts)** Records who did what (hide, delete, report decision, category or role change) for the moderation history.

Q47	`from("moderation_log").select("*") .order("created_at", desc).limit(300)`
	
	`from("users").select("id, display_name") .in("id", ids)`
		`SELECT ml.*, u.display_name AS by_name FROM moderation_log ml LEFT JOIN users u ON u.id = ml.actor_id ORDER BY ml.created_at DESC LIMIT 300;`	**GET /report/history (moderators)** The "History" tab on the Moderation page: the latest 300 actions, filterable by type and moderator.

Q48	`-- fallback when moderation_log does not exist from("reports").select("post_id, status, reviewed_by, reviewed_at, post:posts(title)") .not("reviewed_by", "is", null) .neq("status", "Open")`
	
	`from("posts").select("id, title, hidden_by, hidden_at, hidden_reason") .eq("is_hidden", true) .not("hidden_by", "is", null)`
		`SELECT r.post_id, r.status, r.reviewed_by, r.reviewed_at, p.title FROM reports r LEFT JOIN posts p ON p.id = r.post_id WHERE r.reviewed_by IS NOT NULL AND r.status <> 'Open';`
		
		`SELECT id, title, hidden_by, hidden_at, hidden_reason FROM posts WHERE is_hidden = true AND hidden_by IS NOT NULL;`
			GET /report/history, before the moderation_log table was created Rebuilds a partial history from report decisions and hidden posts. The page then shows a notice that the history is incomplete.

Categories and roles (moderators)			

Q49	`from("categories").select("*")`
	
	`.order("priority").order("category_name")`
	
	`from("post_categories").select("category_id")`
	
	`from("user_interests").select("category_id")`
		`SELECT c.*, (SELECT COUNT(*) FROM post_categories pc WHERE pc.category_id = c.id) AS post_count, (SELECT COUNT(*) FROM user_interests ui WHERE ui.category_id = c.id) AS follower_count FROM categories c ORDER BY c.priority, c.category_name;`	**GET /admin/categories** The category manager on the Moderation page, with how many posts and followers each category has.

Q50	`from("categories").select("id") .ilike("category_name", name)`
	
	`from("categories").insert(value) .select().single()`
		`SELECT id FROM categories WHERE category_name ILIKE :name; -- name already used?`
		
		`INSERT INTO categories (category_name, description, priority) VALUES (:name, :description, :priority) RETURNING *;`
			POST /admin/categories Adds a category. A duplicate name (ignoring case) is refused. The new category appears in the sidebar for everyone.

#	Supabase SDK	SQL equivalent code	What it does and how it affects the website
Q51	`from("categories").update( value) .eq("id", id).select().maybeSingle()`	`UPDATE categories SET category_name = :name, description = :description, priority = :priority WHERE id = :category_id RETURNING *;`	**PUT /admin/categories/:catId** Renames a category or changes its description or priority (its place in the list). Posts show the new name.

Q52	`from("post_categories").se lect("*", { count: "exact", head: true }) .eq("category_id", id)`
	
	`from("user_interests").del ete() .eq("category_id", id)`
	
	`from("categories").delete( ).eq("id", id) .select("id, category_name")`
		`SELECT COUNT(*) FROM post_categories WHERE category_id = :category_id; -- must be 0`
		
		`DELETE FROM user_interests WHERE category_id = :category_id;`
		
		`DELETE FROM categories WHERE id = :category_id RETURNING id, category_name;`
			DELETE /admin/categories/:catId Deletes a category that no post uses, and removes it from everyone's followed list. A category still in use is refused.

Q53	`from("users").select("id, display_name, email, is_mod, is_cmu_p, created_at")`
	
	`.or("display_name.ilike.%q %,email.ilike.%q%") .order("is_mod", desc).order("display_name" ) .limit(50)`
		`SELECT id, display_name, email, is_mod, is_cmu_p, created_at FROM users WHERE display_name ILIKE '%' || :q || '%' OR email ILIKE '%' || :q || '%' ORDER BY is_mod DESC, display_name LIMIT 50;`	**GET /admin/users** Searches users on the Roles screen; moderators are listed first.

Q54	`from("users").select("is_c mu_p") .eq("id", uid).maybeSingle()`
	
	`from("users").select("*", { count: "exact", head: true }).eq("is_mod", true)`
	
	`from("users").update({ is_mod }) .eq("id", uid) .select("id, display_name, is_mod")`
		`-- granting: the user must be CMU Personnel SELECT is_cmu_p FROM users WHERE id = :user_id;`
		
		`-- revoking: at least one moderator must remain SELECT COUNT(*) FROM users WHERE is_mod = true;`
		
		`UPDATE users SET is_mod = :is_mod WHERE id = :user_id RETURNING id, display_name, is_mod;`
			PUT /admin/users/:uid/role Makes a user a moderator or removes the role. That user's screen updates at once: the Moderation tab appears or disappears.

Live updates			
Q55	`supabase.channel("wnm-live ") .httpSend("changed", { topics, post_id }) (frontend: .channel("wnm-live") .on("broadcast", ...))`	`-- No table is read or written. The closest SQL is SELECT realtime.send( jsonb_build_object('topics', :topics, 'post_id', :post_id), 'changed', -- event 'wnm-live', -- topic false -- public channel );`	**After every successful write (live.ts, useLiveUpdates.ts)** Tells every open browser what changed (posts, comments, reports, notifications...). They reload that part through the API, so new posts and comments appear without refreshing the page.

RPC Function (PLPGSQL)

react_to_post

```
DROP FUNCTION react_to_post(uuid,uuid,text);

CREATE OR REPLACE FUNCTION react_to_post(pid uuid, uid UUID, rtype text,
change_reaction boolean)
RETURNS INTEGER
SECURITY DEFINER SET search_path = "public"
AS $$
DECLARE
  new_total_reactions int8;
BEGIN
  if change_reaction then
    update post_reactions set reaction_type = rtype::reaction_type
    where post_reactions.post_id = pid AND post_reactions.user_id = uid;
  else
    WITH deleted AS (
      DELETE FROM post_reactions
      WHERE post_reactions.post_id = pid AND post_reactions.user_id = uid
      RETURNING *
    )
    INSERT INTO post_reactions (post_id, user_id, reaction_type)
    SELECT pid, uid, rtype::reaction_type
    WHERE NOT EXISTS (SELECT 1 FROM deleted);
  end if;

  SELECT COUNT(*) INTO new_total_reactions FROM post_reactions WHERE
post_reactions.post_id = pid;
  RETURN new_total_reactions;
END;
$$ LANGUAGE plpgsql;

-- 1. First, stop EVERYONE from being able to call it
REVOKE EXECUTE ON FUNCTION react_to_post(pid uuid, uid UUID, rtype text,
change_reaction boolean) FROM PUBLIC;
REVOKE EXECUTE ON FUNCTION react_to_post(pid uuid, uid UUID, rtype text,
change_reaction boolean) FROM anon;

-- 2. Then, only allow 'authenticated' users (logged-in users) to call it
```

```
GRANT EXECUTE ON FUNCTION react_to_post(pid uuid, uid UUID, rtype text,
change_reaction boolean) TO authenticated;
```

react_to_comment

```
DROP FUNCTION react_to_comment(uuid,uuid,text);

CREATE OR REPLACE FUNCTION react_to_comment(cid uuid, uid UUID, rtype text,
change_reaction boolean)
RETURNS INTEGER
SECURITY DEFINER SET search_path = "public"
AS $$
DECLARE
  new_total_reactions int8;
BEGIN
  if change_reaction then
    update comment_reactions set reaction_type = rtype::reaction_type
    where comment_reactions.comment_id = cid AND comment_reactions.user_id =
uid;
  else
    WITH deleted AS (
      DELETE FROM comment_reactions
      WHERE comment_reactions.comment_id = cid AND comment_reactions.user_id =
uid
      RETURNING *
    )
    INSERT INTO comment_reactions (comment_id, user_id, reaction_type)
    SELECT cid, uid, rtype::reaction_type
    WHERE NOT EXISTS (SELECT 1 FROM deleted);
  end if;

  SELECT COUNT(*) INTO new_total_reactions FROM comment_reactions WHERE
comment_reactions.comment_id = cid;
  RETURN new_total_reactions;
END;
$$ LANGUAGE plpgsql;

-- 1. First, stop EVERYONE from being able to call it
REVOKE EXECUTE ON FUNCTION react_to_comment(cid uuid, uid UUID, rtype text,
change_reaction boolean) FROM PUBLIC;
REVOKE EXECUTE ON FUNCTION react_to_comment(cid uuid, uid UUID, rtype text,
change_reaction boolean) FROM anon;
```

```
-- 2. Then, only allow 'authenticated' users (logged-in users) to call it
GRANT EXECUTE ON FUNCTION react_to_comment(cid uuid, uid UUID, rtype text,
change_reaction boolean) TO authenticated;
```

insert_notification (trigger)

```
create or replace function public.insert_notification()
returns trigger as $$
begin
  insert into public.notifications (user_id, post_id, message, is_read)
  select
    id,
    new.id,
    new.message, --send full, filter in frontend/backend, or should i filter
here?
    false
  from public.users;

  return new;
end;
$$ language plpgsql security definer set search_path = "public";

create or replace trigger on_announcement_post
  after insert on public.posts
  for each row when (new.is_important = true) execute procedure
public.insert_notification()
```

user_interest_op

```
-- DROP FUNCTION user_interests_op(uuid,uuid);

CREATE OR REPLACE FUNCTION user_interests_op(uid UUID, cate_id uuid[])
RETURNS SETOF user_interests
SECURITY DEFINER SET search_path = "public"
AS $$
BEGIN
  delete from user_interests where user_id = uid;

  INSERT INTO user_interests (user_id, category_id)
  select uid, unnest(cate_id);

  RETURN QUERY SELECT * FROM user_interests WHERE user_id = uid;
```

```
END;
$$ LANGUAGE plpgsql;

-- 1. First, stop EVERYONE from being able to call it
REVOKE EXECUTE ON FUNCTION user_interests_op(uid UUID, cate_id uuid[]) FROM
PUBLIC;
REVOKE EXECUTE ON FUNCTION user_interests_op(uid UUID, cate_id uuid[]) FROM
anon;

-- 2. Then, only allow 'authenticated' users (logged-in users) to call it
GRANT EXECUTE ON FUNCTION user_interests_op(uid UUID, cate_id uuid[]) TO
authenticated;
```

update_post_cate

```
-- DROP FUNCTION user_interests_op(uuid,uuid,text);

CREATE OR REPLACE FUNCTION update_post_cate(pid uuid, cate_name text[])
RETURNS SETOF post_categories
SECURITY DEFINER SET search_path = "public"
AS $$
BEGIN
  delete from post_categories where post_id = pid;

  INSERT INTO post_categories (post_id, category_id)
  select pid, c.id from categories c where c.category_name = any(cate_name);

  RETURN QUERY SELECT * FROM post_categories WHERE post_id = pid;
END;
$$ LANGUAGE plpgsql;

-- 1. First, stop EVERYONE from being able to call it
REVOKE EXECUTE ON FUNCTION update_post_cate(pid uuid, cate_id text[]) FROM
PUBLIC;
REVOKE EXECUTE ON FUNCTION update_post_cate(pid uuid, cate_id text[]) FROM
anon;

-- 2. Then, only allow 'authenticated' users (logged-in users) to call it
GRANT EXECUTE ON FUNCTION update_post_cate(pid uuid, cate_id text[]) TO
authenticated;
```

Other functions and triggers

```
-- BR20: a comment's created_at cannot be earlier than its post's created_at.
-- This is a cross-table rule, so it needs a trigger (a plain CHECK can't
-- reference another table).
create or replace function fn_check_comment_after_post()
returns trigger as $$
declare
    v_post_created_at timestamptz;
begin
    select created_at into v_post_created_at from posts where id = new.post_id;
    if new.created_at < v_post_created_at then
        raise exception 'BR20 violation: comment.created_at (%) cannot be
earlier than its post.created_at (%)',
            new.created_at, v_post_created_at;
    end if;
    return new;
end;
$$ language plpgsql;

create trigger trg_check_comment_after_post
    before insert or update on comments
    for each row execute function fn_check_comment_after_post();

-- BR12: every post must have at least one tag/category.
-- A plain CHECK/NOT NULL can't express "at least one row exists in a
-- child table", so this is enforced with a deferred constraint trigger:
-- it fires at end-of-transaction, so you can INSERT the post and its
-- post_categories row(s) in the same transaction and still pass.
create or replace function fn_check_post_has_category()
returns trigger as $$
begin
    if not exists (select 1 from post_categories where post_id = new.id) then
        raise exception 'BR12 violation: post % must have at least one
category/tag', new.id;
    end if;
    return null;
end;
$$ language plpgsql;

create constraint trigger trg_check_post_has_category
    after insert on posts
```

```
    deferrable initially deferred
    for each row execute function fn_check_post_has_category();

-- BR22: only a Moderator can change a report's status (i.e. set reviewed_by).
create or replace function fn_check_reviewer_is_moderator()
returns trigger as $$
declare
    v_is_mod boolean;
begin
    if new.reviewed_by is not null then
        select is_mod into v_is_mod from users where id = new.reviewed_by;
        if v_is_mod is not true then
            raise exception 'BR22 violation: user % is not a Moderator and
cannot review a report', new.reviewed_by;
        end if;
    end if;
    return new;
end;
$$ language plpgsql;

create trigger trg_check_reviewer_is_moderator
    before insert or update on reports
    for each row execute function fn_check_reviewer_is_moderator();



-- ============================================================================
-- 13. updated_at bookkeeping for posts
-- ============================================================================
create or replace function fn_set_updated_at()
returns trigger as $$
begin
    new.updated_at = now();
    return new;
end;
$$ language plpgsql;

create trigger trg_posts_set_updated_at
    before update on posts
    for each row execute function fn_set_updated_at();
```

Miscellaneous

RLS Policies

```
-- ============================================================================
-- WhongNaiMor — Row Level Security (RLS)
-- ============================================================================
-- Supabase requires RLS to be turned on for any table exposed through
-- its auto-generated REST/GraphQL API, or the table is simply
-- unreachable from the client. Since this MVP has NO real CMU OAuth
-- login yet (per the proposal's "out of scope" section — everyone
-- browses as a template/anonymous account), these policies are
-- intentionally permissive: read is open to everyone, write is open
-- to any authenticated-looking request from the backend service role.
--
-- >>> TIGHTEN THESE once CMU OAuth (Supabase Auth) is wired up. <<<
-- The comments below show what each policy should become at that point.
-- ============================================================================

alter table users              enable row level security;
alter table categories         enable row level security;
alter table posts              enable row level security;
alter table comments           enable row level security;
alter table media              enable row level security;
alter table post_reactions     enable row level security;
alter table comment_reactions  enable row level security;
alter table user_interests     enable row level security;
alter table post_categories    enable row level security;
alter table reports            enable row level security;
alter table notifications      enable row level security;

-- ----------------------------------------------------------------------------
-- Public read access (anyone, including Non-CMU visitors, can browse)
-- ----------------------------------------------------------------------------
create policy "public can read users"             on users             for
select using (true);
create policy "public can read categories"        on categories        for
select using (true);
create policy "public can read posts"             on posts             for
select using (true);
```

```sql
create policy "public can read comments"          on comments          for
select using (true);
create policy "public can read media"             on media             for
select using (true);
create policy "public can read post_categories"   on post_categories   for
select using (true);
create policy "public can read post_reactions"    on post_reactions    for
select using (true);
create policy "public can read comment_reactions" on comment_reactions for
select using (true);

-- Reports and notifications are private-ish: expose read for the MVP
-- demo, but this is the first thing to lock down (see note at bottom).
create policy "public can read reports"       on reports       for select using
(true);
create policy "public can read notifications" on notifications for select using
(true);
create policy "public can read user_interests" on user_interests for select
using (true);

-- ------------------------------------------------------------------
-- Write access — MVP: allow all writes (no auth to key off yet).
-- The backend Express API is the actual gatekeeper for who can post,
-- react, report, etc. during MVP; the DB layer stays open so seed
-- scripts and the demo (single "logged-in" template user) work.
-- ------------------------------------------------------------------
create policy "anyone can write posts"            on posts             for all
using (true) with check (true);
create policy "anyone can write comments"         on comments          for all
using (true) with check (true);
create policy "anyone can write media"            on media             for all
using (true) with check (true);
create policy "anyone can write post_reactions"   on post_reactions    for all
using (true) with check (true);
create policy "anyone can write comment_reactions" on comment_reactions for all
using (true) with check (true);
create policy "anyone can write user_interests"   on user_interests    for all
using (true) with check (true);
create policy "anyone can write post_categories"  on post_categories   for all
using (true) with check (true);
create policy "anyone can write reports"          on reports           for all
using (true) with check (true);
```

```
create policy "anyone can write notifications"     on notifications     for all
using (true) with check (true);
create policy "anyone can write users"             on users             for all
using (true) with check (true);
create policy "anyone can write categories"        on categories        for all
using (true) with check (true);

-- ============================================================
-- FUTURE (post-MVP, once auth.uid() maps to users.id via CMU OAuth):
--
--   -- only the author can edit/delete their own post
--   create policy "owner can update own post" on posts
--       for update using (auth.uid() = user_id);
--
--   -- only a moderator can change report status
--   create policy "moderator can review reports" on reports
--       for update using (
--           exists (select 1 from users u where u.id = auth.uid() and
u.is_mod)
--       );
--
--   -- a user can only mark their own notifications as read
--   create policy "owner can update own notification" on notifications
--       for update using (auth.uid() = user_id);
-- ============================================================
```

Data Sampling

```
-- ============================================================
-- WhongNaiMor — Sample seed data
-- Fake data only (PDPA-compliant, per Lab 6's note on personal data).
-- Wrapped in one transaction because BR12 ("every post needs >=1 tag")
-- is enforced by a DEFERRED constraint trigger — the post and its
-- post_categories row(s) must land in the same transaction.
-- ============================================================

begin;

-- ------------------------------------------------------------
-- Categories (admin/dev-seeded, per the proposal's note)
```

```sql
-- ------------------------------------------------------------------------
insert into categories (id, category_name, description, priority) values
    ('11111111-1111-1111-1111-111111111111', 'News',          'General CMU
news',                 3),
    ('11111111-1111-1111-1111-111111111112', 'Event',         'Campus events
and activities',      2),
    ('11111111-1111-1111-1111-111111111113', 'Question',      'Questions asked
by students',       2),
    ('11111111-1111-1111-1111-111111111114', 'Lost and Found','Lost/found items
around campus',    2),
    ('11111111-1111-1111-1111-111111111115', 'Warning',       'Safety warnings
/ alerts',           1);

-- ------------------------------------------------------------------------
-- Users: 3 students, 1 CMU-personnel-only, 1 moderator
-- ------------------------------------------------------------------------
insert into users (id, cmu_account, display_name, email, is_cmu_p, is_mod)
values
    ('22222222-2222-2222-2222-222222222221', 'ploy_s@cmu.ac.th',  'Ploy
Sriwan',  'ploy_s@cmu.ac.th',  true,  false),
    ('22222222-2222-2222-2222-222222222222', 'nat_w@cmu.ac.th',   'Nat Wong',
'nat_w@cmu.ac.th',   true,  false),
    ('22222222-2222-2222-2222-222222222223', 'mook_c@cmu.ac.th',  'Mook Chai',
'mook_c@cmu.ac.th',  true,  false),
    ('22222222-2222-2222-2222-222222222224', 'somchai_j@cmu.ac.th','Dr.
Somchai', 'somchai_j@cmu.ac.th', true, false),
    ('22222222-2222-2222-2222-222222222225', 'admin_mod@cmu.ac.th','WhongNaiMor
Admin', 'admin_mod@cmu.ac.th', true, true);

-- ------------------------------------------------------------------------
-- Posts (+ required category tag in the same transaction)
-- ------------------------------------------------------------------------
insert into posts (id, user_id, content, is_important) values
    ('33333333-3333-3333-3333-333333333331',
'22222222-2222-2222-2222-222222222225',
     'Library will close early (5 PM) this Friday for maintenance.', true),
    ('33333333-3333-3333-3333-333333333332',
'22222222-2222-2222-2222-222222222221',
     'Anyone know a good spot for cheap boba near the Art Center?', false),
    ('33333333-3333-3333-3333-333333333333',
'22222222-2222-2222-2222-222222222222',
     'Found a blue water bottle near the engineering building, DM me!', false);
```

```
insert into post_categories (post_id, category_id) values
    ('33333333-3333-3333-3333-333333333331',
'11111111-1111-1111-1111-111111111115'), -- Warning
    ('33333333-3333-3333-3333-333333333332',
'11111111-1111-1111-1111-111111111113'), -- Question
    ('33333333-3333-3333-3333-333333333333',
'11111111-1111-1111-1111-111111111114'); -- Lost and Found

-- ------------------------------------------------------------------------
-- Comments (with one threaded reply)
-- ------------------------------------------------------------------------
insert into comments (id, post_id, user_id, parent_comment_id, content) values
    ('44444444-4444-4444-4444-444444444441',
'33333333-3333-3333-3333-333333333332',
     '22222222-2222-2222-2222-222222222223', null, 'Try the one next to the
dorm gate, it''s cheap!'),
    ('44444444-4444-4444-4444-444444444442',
'33333333-3333-3333-3333-333333333332',
     '22222222-2222-2222-2222-222222222221',
'44444444-4444-4444-4444-444444444441', 'Thank you! Will check it out.');

-- ------------------------------------------------------------------------
-- Reactions
-- ------------------------------------------------------------------------
insert into post_reactions (post_id, user_id, reaction_type) values
    ('33333333-3333-3333-3333-333333333331',
'22222222-2222-2222-2222-222222222221', 'care'),
    ('33333333-3333-3333-3333-333333333331',
'22222222-2222-2222-2222-222222222222', 'like'),
    ('33333333-3333-3333-3333-333333333332',
'22222222-2222-2222-2222-222222222223', 'like');

insert into comment_reactions (comment_id, user_id, reaction_type) values
    ('44444444-4444-4444-4444-444444444441',
'22222222-2222-2222-2222-222222222221', 'like');

-- ------------------------------------------------------------------------
-- User interests (followed categories)
-- ------------------------------------------------------------------------
insert into user_interests (user_id, category_id) values
```

```
    ('22222222-2222-2222-2222-222222222221',
'11111111-1111-1111-1111-111111111112'),
    ('22222222-2222-2222-2222-222222222222',
'11111111-1111-1111-1111-111111111115');

-- ------------------------------------------------------------------------
-- A report + a moderator reviewing it (BR22: reviewer must be is_mod)
-- ------------------------------------------------------------------------
insert into reports (id, post_id, user_id, reason, status, reviewed_by,
reviewed_at) values
    ('55555555-5555-5555-5555-555555555551',
'33333333-3333-3333-3333-333333333333',
     '22222222-2222-2222-2222-222222222224', 'Possible duplicate of an earlier
lost & found post',
     'Reviewed', '22222222-2222-2222-2222-222222222225', now());

-- ------------------------------------------------------------------------
-- Notification fan-out for the Announcement post
-- ------------------------------------------------------------------------
insert into notifications (user_id, post_id, message, is_read) values
    ('22222222-2222-2222-2222-222222222221',
'33333333-3333-3333-3333-333333333331',
     'New announcement: Library closes early this Friday.', false),
    ('22222222-2222-2222-2222-222222222222',
'33333333-3333-3333-3333-333333333331',
     'New announcement: Library closes early this Friday.', true);

Commit;
```

Changes Made Later

2026-10-13_hide_posts.sql

```
-- Hide posts instead of deleting them.
-- A deleted post takes its reports (and the reporters' notifications) with it,
or fails
-- on the foreign key, so moderators could not close the reports. A hidden post
keeps
-- them, disappears from the feed, and stays visible to its author with the
reason.
--
```

```sql
-- Run once in Supabase -> SQL Editor (project dcqzspcxgwcbrogcxyhm). Safe to
run again.

alter table public.posts add column if not exists is_hidden     boolean not
null default false;
alter table public.posts add column if not exists hidden_at     timestamptz;
alter table public.posts add column if not exists hidden_by     uuid;
alter table public.posts add column if not exists hidden_reason text;

create index if not exists posts_hidden_idx on public.posts (id) where
is_hidden;

-- Make the API (PostgREST) see the new columns right away.
notify pgrst, 'reload schema';

-- Check: should list the four columns.
select column_name, data_type, column_default
from information_schema.columns
where table_schema = 'public' and table_name = 'posts' and column_name like
'%hidden%'
order by column_name;
```

2026-10-13_post_edits.sql

```sql
-- Remember that a post was edited, and by whom, so the "edited" / "edited by a
moderator"
-- label survives a reload. Run once in Supabase -> SQL Editor. Safe to run
again.
alter table public.posts add column if not exists edited_at timestamptz;
alter table public.posts add column if not exists edited_by uuid;
notify pgrst, 'reload schema';
```

2026-10-13_fix_announcement_notifications.sql

```sql
-- Fix: creating an announcement failed with `column reference "id" is
ambiguous`.
--
-- The old trigger (on_announcement_post -> insert_notification) had three
problems:
--   1. `ui.user_id = id` -- `id` could be post_categories.id or
user_interests.id (the error).
```

```sql
--   2. It ran right after the posts insert, before the API adds the post's
categories,
--      so the interest check could never find anything.
--   3. Had it matched, it would have notified every user, not only the
interested ones.
--
-- New behaviour: when an announcement gets a category, every user interested
in that
-- category is notified once (not the author, never twice for the same post).
--
-- Run once in Supabase -> SQL Editor (project dcqzspcxgwcbrogcxyhm). Safe to
run again.

create or replace function public.notify_announcement_followers()
returns trigger
language plpgsql
security definer
set search_path = public
as $$
begin
  insert into public.notifications (user_id, post_id, message, is_read)
  select distinct
    ui.user_id,
    p.id,
    'ประกาศใหม่: ' || coalesce(nullif(p.title, ''), left(p.content, 80)),
    false
  from public.posts as p
  join public.user_interests as ui on ui.category_id = new.category_id
  where p.id = new.post_id
    and p.is_important
    and ui.user_id <> p.user_id
    and not exists (
      select 1 from public.notifications as n
      where n.post_id = p.id and n.user_id = ui.user_id
    );
  return new;
end;
$$;

-- Notify from post_categories, where the categories actually arrive.
drop trigger if exists on_announcement_post on public.posts;
drop trigger if exists on_announcement_category on public.post_categories;
```

```sql
create trigger on_announcement_category
  after insert on public.post_categories
  for each row execute function public.notify_announcement_followers();

-- Check: should show on_announcement_category on post_categories, and nothing
named on_announcement_post.
select tgname as trigger_name, tgrelid::regclass as on_table
from pg_trigger
where tgname in ('on_announcement_post', 'on_announcement_category');
```

2026-10-13_lock_down_tables.sql

```sql
-- Security: the browser only uses Supabase for login and realtime broadcast;
all data goes
-- through the API, which uses the service role. So the public (anon) and
signed-in
-- (authenticated) roles need no direct access to tables, views or functions.
-- Before this, anyone with the anon key (it ships in the web page) could read
every
-- report and every user's notifications. Run once in Supabase -> SQL Editor.
Safe to rerun.

revoke all on all tables    in schema public from anon, authenticated;
revoke all on all sequences in schema public from anon, authenticated;
revoke execute on all functions in schema public from anon, authenticated;

-- Keep new tables/functions closed too (Supabase grants them to anon by
default).
alter default privileges in schema public revoke all on tables    from anon,
authenticated;
alter default privileges in schema public revoke all on sequences from anon,
authenticated;
alter default privileges in schema public revoke execute on functions from
anon, authenticated;

-- Row level security as a second lock (the API's service role bypasses it).
do $$
declare t text;
begin
  for t in select tablename from pg_tables where schemaname = 'public' loop
    execute format('alter table public.%I enable row level security', t);
  end loop;
```

```
end $$;

-- Old announcement trigger function, replaced by
notify_announcement_followers().
drop function if exists public.insert_notification();

notify pgrst, 'reload schema';
```

2026-10-13_moderation_log.sql

```
-- Moderation log: one row per moderator action, so the history page keeps
every decision
-- (reports.reviewed_by only remembers the latest one per report).
-- Written by the API only (service role); closed to anon/authenticated like
every other table.
-- Run once in Supabase -> SQL Editor (project dcqzspcxgwcbrogcxyhm). Safe to
run again.

create table if not exists public.moderation_log (
  id          uuid primary key default gen_random_uuid(),
  actor_id    uuid not null references public.users(id) on delete cascade,
  action      text not null,          -- report_reviewed, report_dismissed,
report_reopened, post_hidden, post_unhidden,
                                      -- post_edited, post_deleted,
comment_deleted, category_created, category_updated,
                                      -- category_deleted, role_granted,
role_revoked
  post_id     uuid,                   -- no foreign key: the entry must outlive
a deleted post
  post_title  text,                   -- kept for the same reason
  target_id   uuid,                   -- the comment, category or user the
action was about
  detail      text,                   -- reason, category name, user name,
number of reports
  created_at  timestamptz not null default now()
);

create index if not exists moderation_log_created_idx on public.moderation_log
(created_at desc);

revoke all on public.moderation_log from anon, authenticated;
alter table public.moderation_log enable row level security;
```

```
notify pgrst, 'reload schema';

-- Check: should return one row named moderation_log.
select table_name from information_schema.tables where table_schema = 'public'
and table_name = 'moderation_log';
```

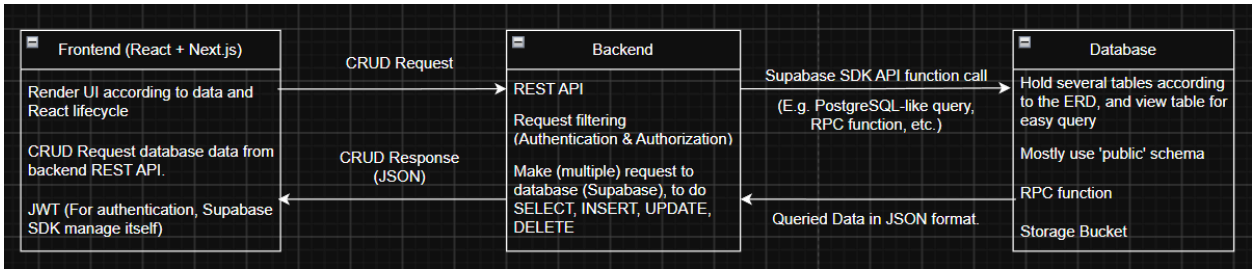
2026-10-13_followers_from_api.sql

```
-- Notifications for followed categories now come from the API
(api/src/followers.ts):
-- every new post in a followed category, not only announcements, with the
category's name,
-- and also when a moderator turns an existing post into an announcement.
-- The old trigger only covered announcements at creation, so it is no longer
needed.
-- Until this runs, the API sees the trigger's notifications and does not send
a second one.
-- Run once in Supabase -> SQL Editor. Safe to run again.

drop trigger if exists on_announcement_category on public.post_categories;
drop trigger if exists on_announcement_post on public.posts;
drop function if exists public.notify_announcement_followers();

-- Check: should return no rows.
select tgname from pg_trigger where tgname in ('on_announcement_category',
'on_announcement_post');
```

Database System Architecture



Technology stack

Layer	Choice	Reason (one line)
Database engine	PostgreSQL 16	Supabase use PostgreSQL (Supabase is the online database service we will use when hosting.)
Modelling tool	Draw.io	Required for this course
Query / admin tool	pgAdmin or Supabase Dashboard	You can actually run queries directly on the Supabase dashboard. pgAdmin becomes the backup option.
Application language (if any)	JavaScript	The main language for all the libraries and frameworks below.
Application framework (if any)	Express, React, Next.js	Express for REST API backend, React and Next.js for frontend.
Hosting / environment	Vercel and Render	Vercel hosts frontend code, Render hosts backend API.
Version control	GitHub	Everyone in the team has experience with Git & GitHub before, and it has CI/CD pipeline with Vercel and Render.

Milestones

Planned

#	Milestone	Deliverable	Due (week)	Owner
M1	Proposal approved	This document	Week 6	All
M2	Database & system design completed	ER Diagram + Database design & Tables in Supabase	15 Sep	Parintorn, Thanwisit
M3	Backend completed	Backend API + Database connection	17 Sep	Thanwisit
M4	Frontend completed	Main pages + UI	18 Sep	Natphisit
M5	System integration & testing	Working prototype + Test results	19 Sep	All
M6	Final project completed	Final system + Report + Demo	20 Sep	All

Note: **Due dates are subject to change within maximum time frame of 2 weeks. (Minimum time frame = 1 week (14 - 21 Sep), maximum time frame = 2 weeks (14 - 28 Sep).)**

Actual

#	Milestone	Deliverable	Due (week)	Owner
M1	Proposal approved	This document	08/09/2026	All
M2	Database & system design completed	ER Diagram + Database design & Tables in Supabase	15/09/2026	Parintorn, Thanwisit
M3	Backend completed	Backend API + Database connection	20/09/2026	Thanwisit
M4	Frontend completed	Main pages + UI	23/09/2026	Natphisit
M5	System integration & testing	Working prototype + Test results	14/10/2026	All

M6	Final project completed	Final system + Report + Demo	14/10/2026	All

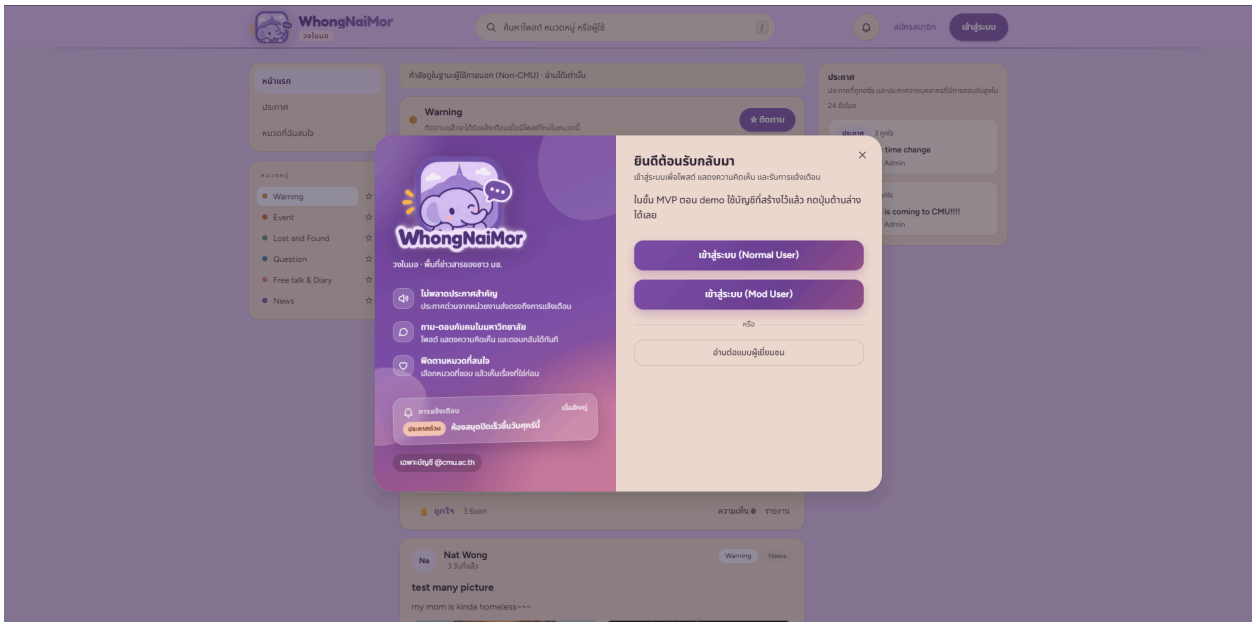
Justification: We couldn't conclude the project within the adjusted timeframe because

- **Everyone took a 1-week break**
- **Different understanding of the business and not communicate well enough.**
- **Feature creep**

UI

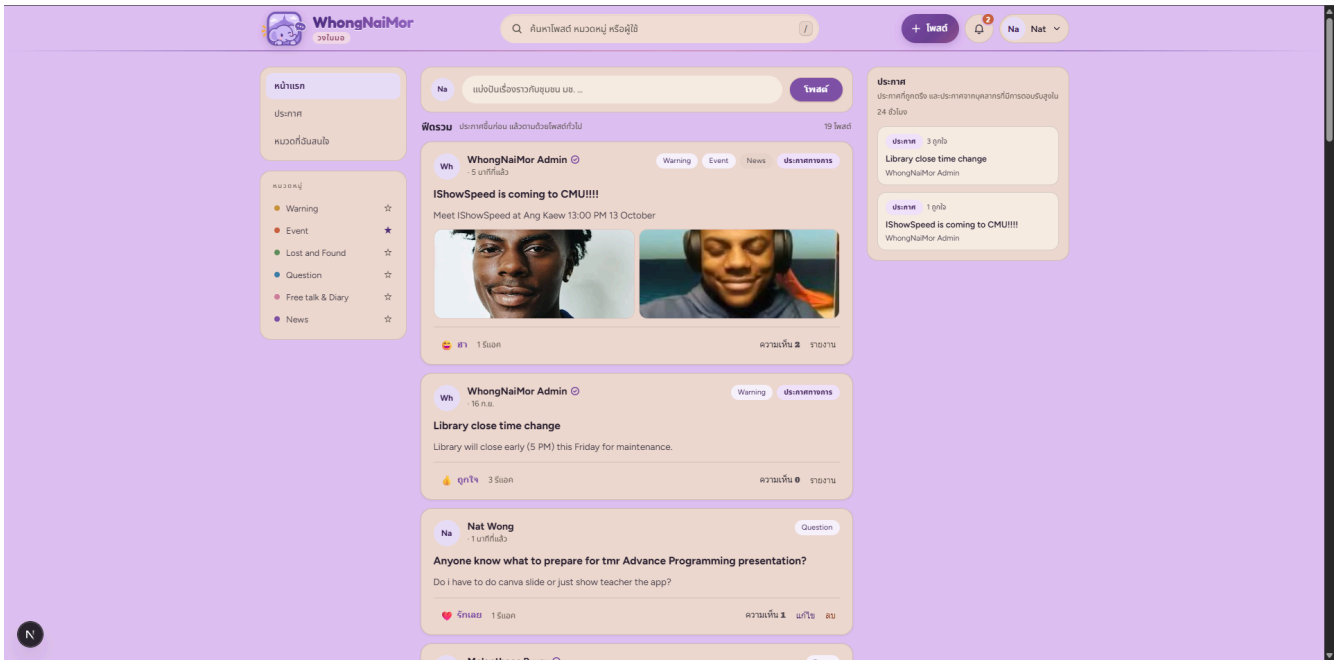
Login

Sign in with a CMU account. For the demo, the two buttons sign in as a prepared normal user or moderator.



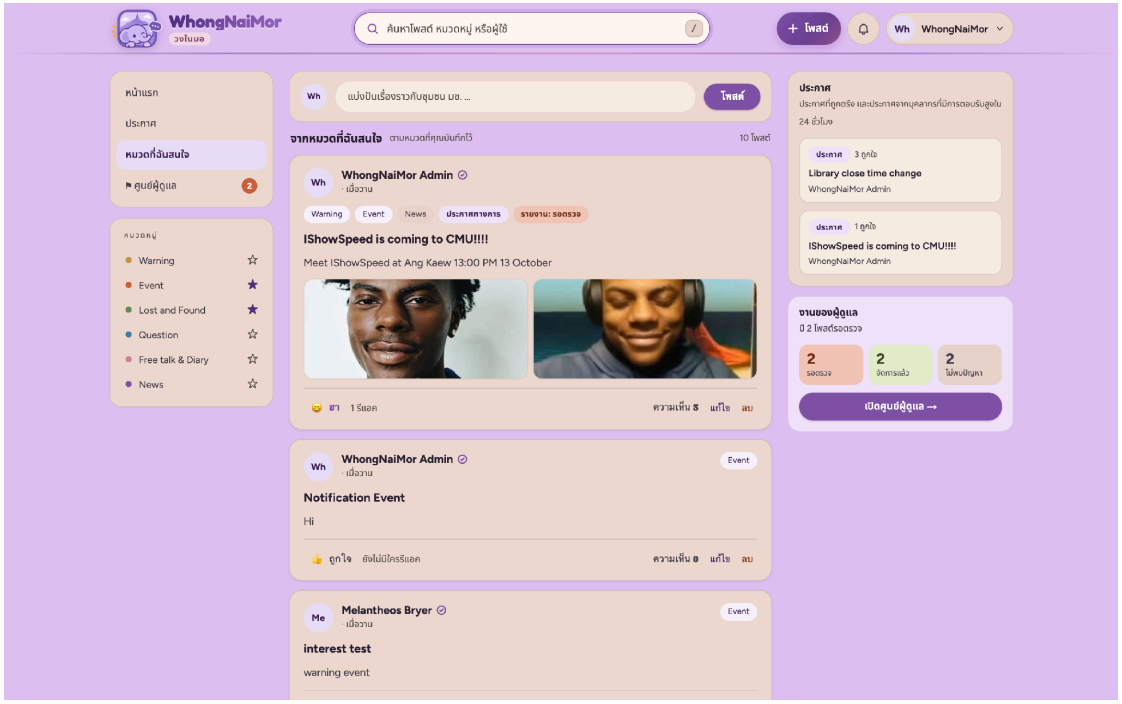
Feed

The home feed: announcements first, then the newest posts, loading more as the user scrolls.



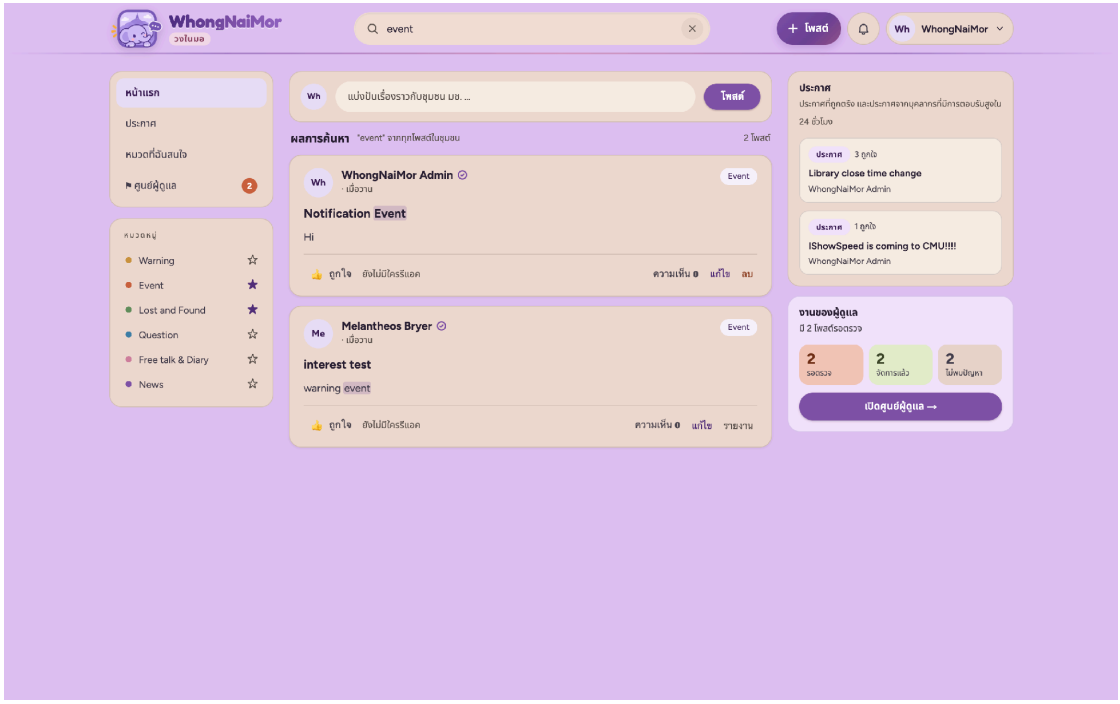
Followed Categories

Starring a category in the sidebar follows it; this tab shows only posts from followed categories, and new posts there send a notification.



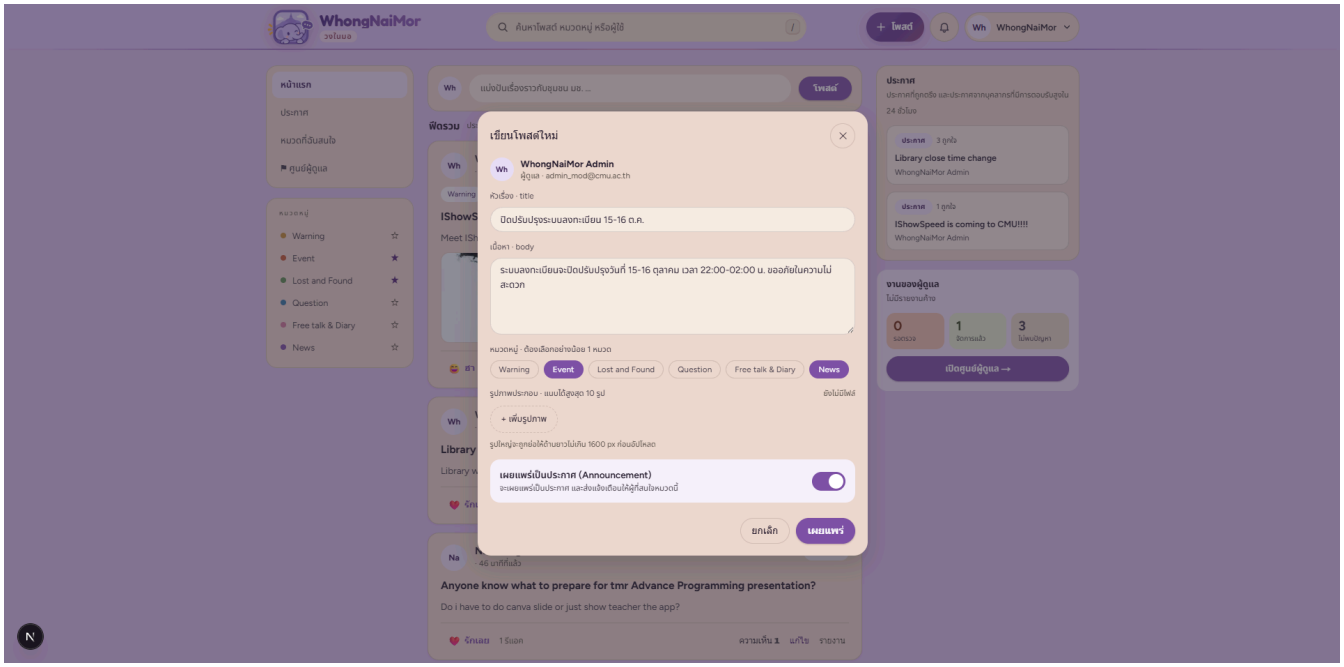
Search

Searching matches post titles, content and author names; the matching words are highlighted.



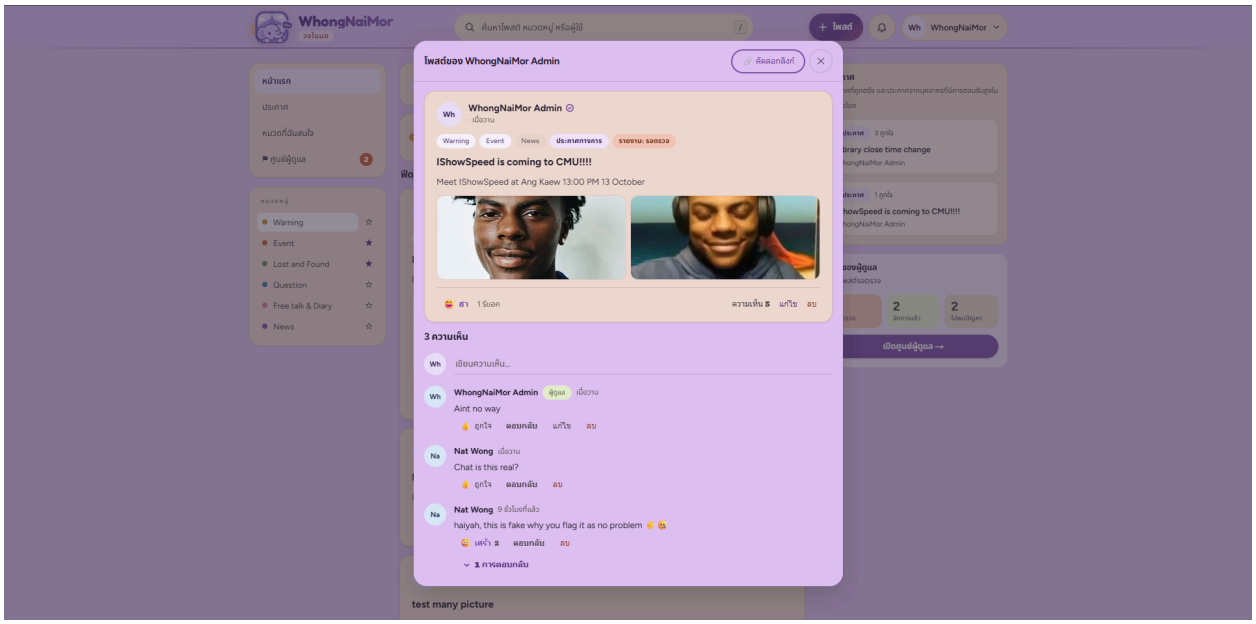
Create Post

Writing a post with a title, one or more categories, and up to 10 photos and 1 video.



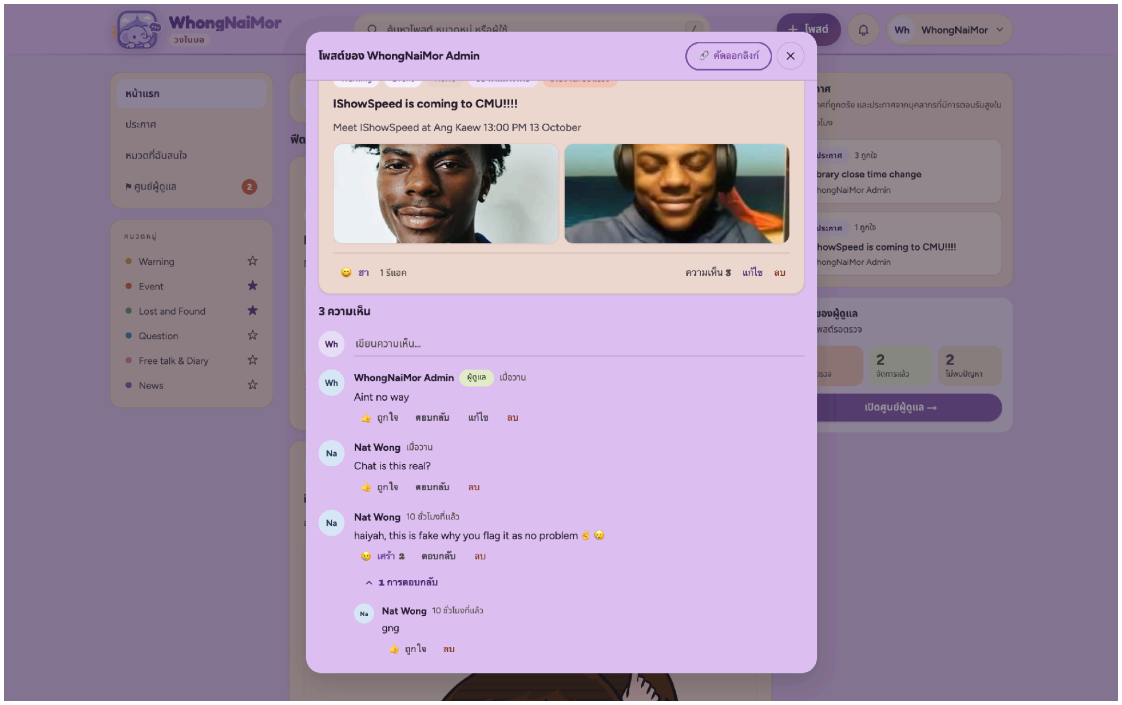
Post

One post opened on its own, from a link or a notification.



Comments and Replies

Comments under a post, with threaded replies; the author and moderators can edit or delete them.



Reactions

Hovering the reaction button (or holding it on a phone) opens the six reactions; clicking the same one again removes it.

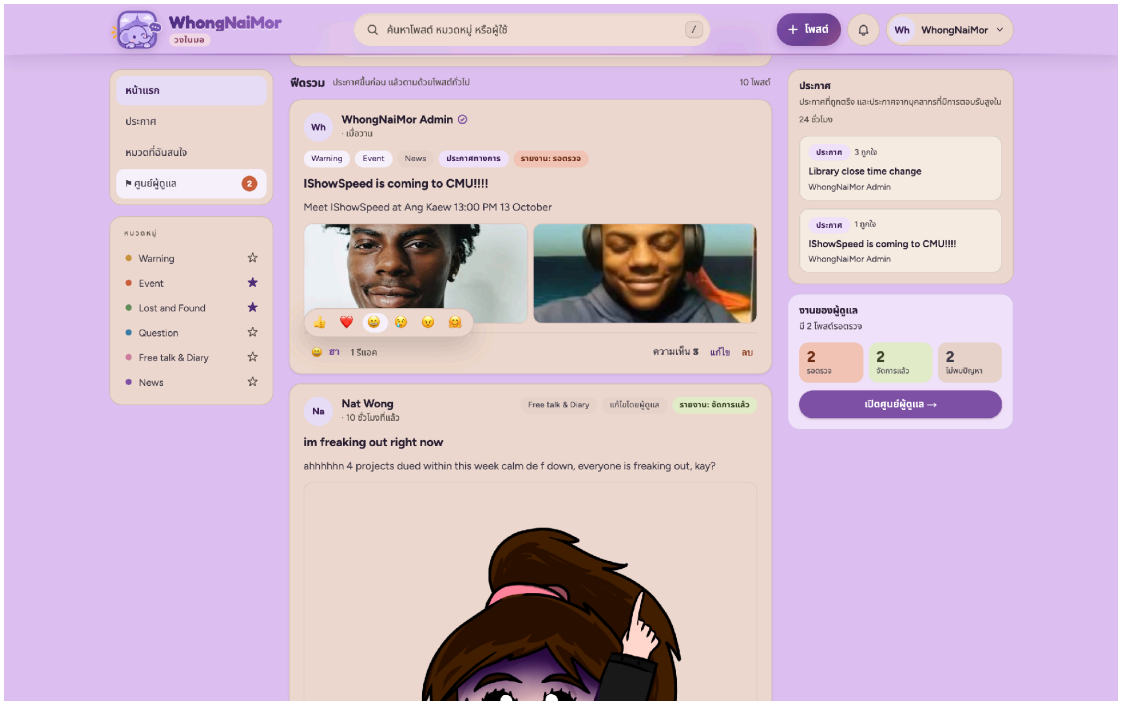
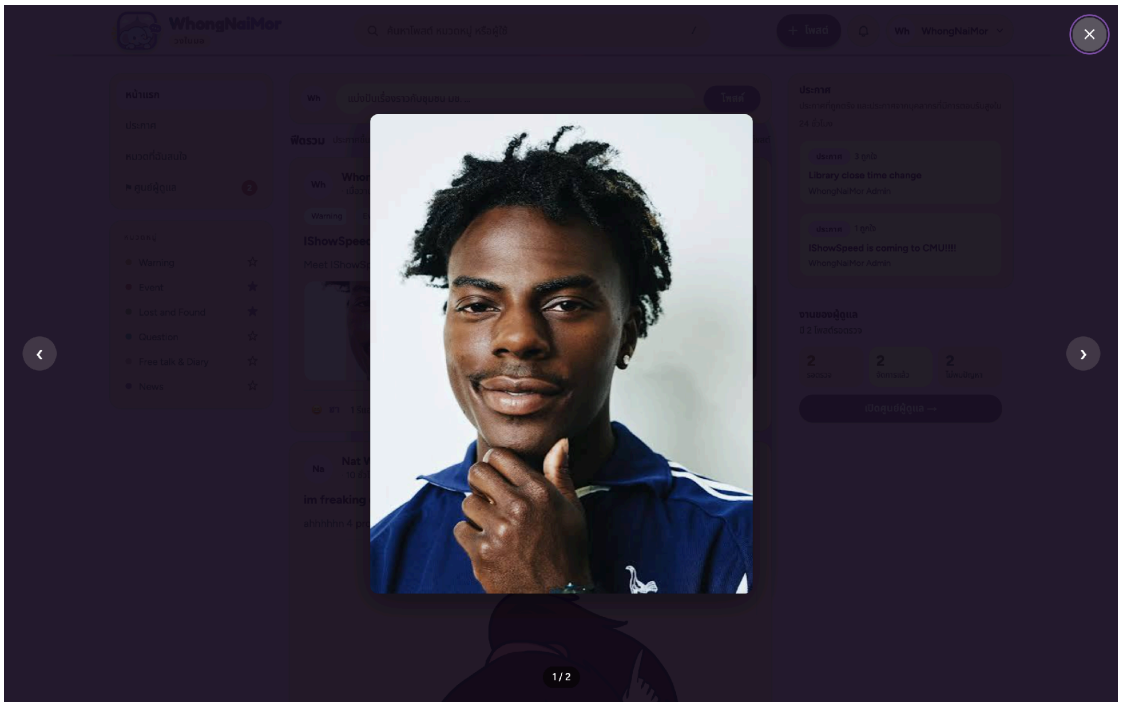


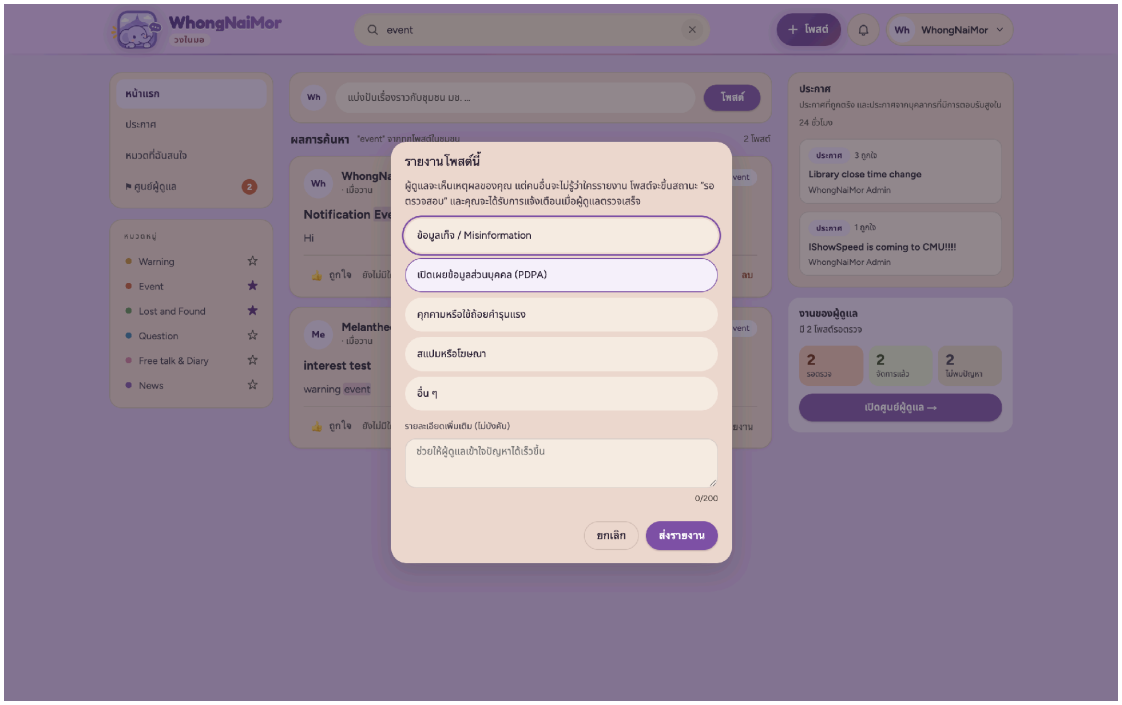
Image Viewer

Photos open full screen, and the arrows or arrow keys move between the photos of the post.



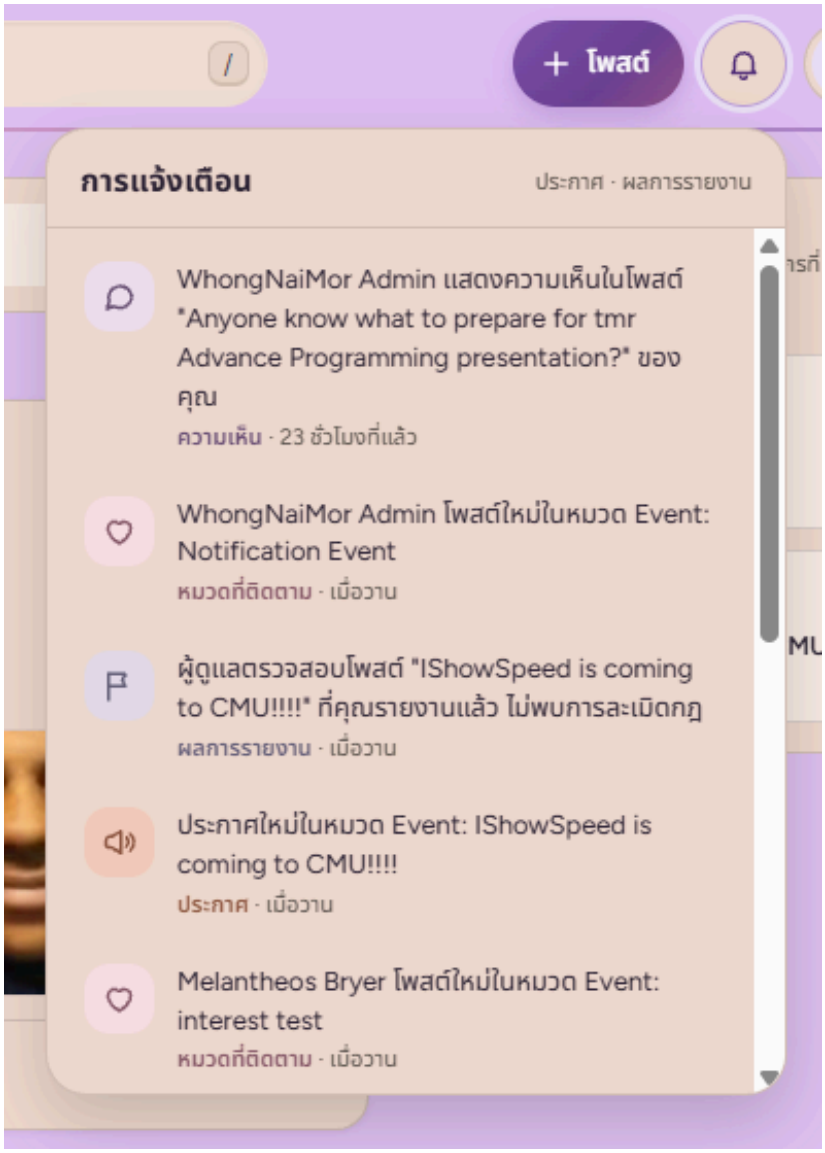
Report a Post

Any signed-in user can report a post with a reason; the post then shows as waiting for review.



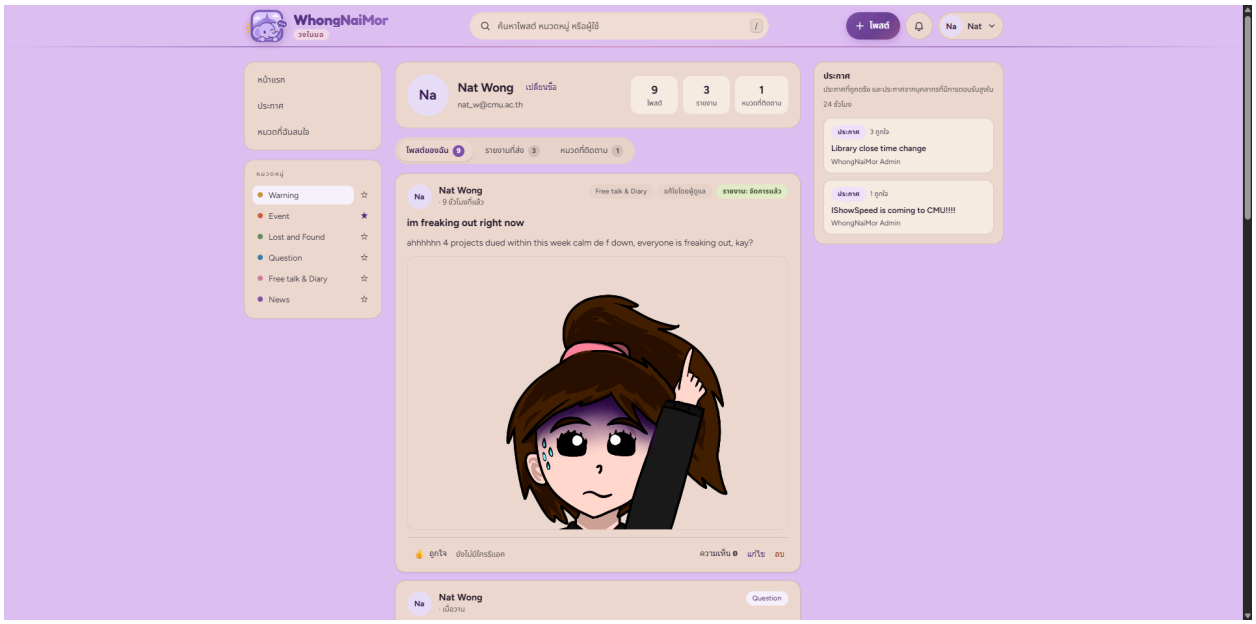
Notification

The bell lists comments, replies, report decisions and new posts in followed categories; unread ones are marked.



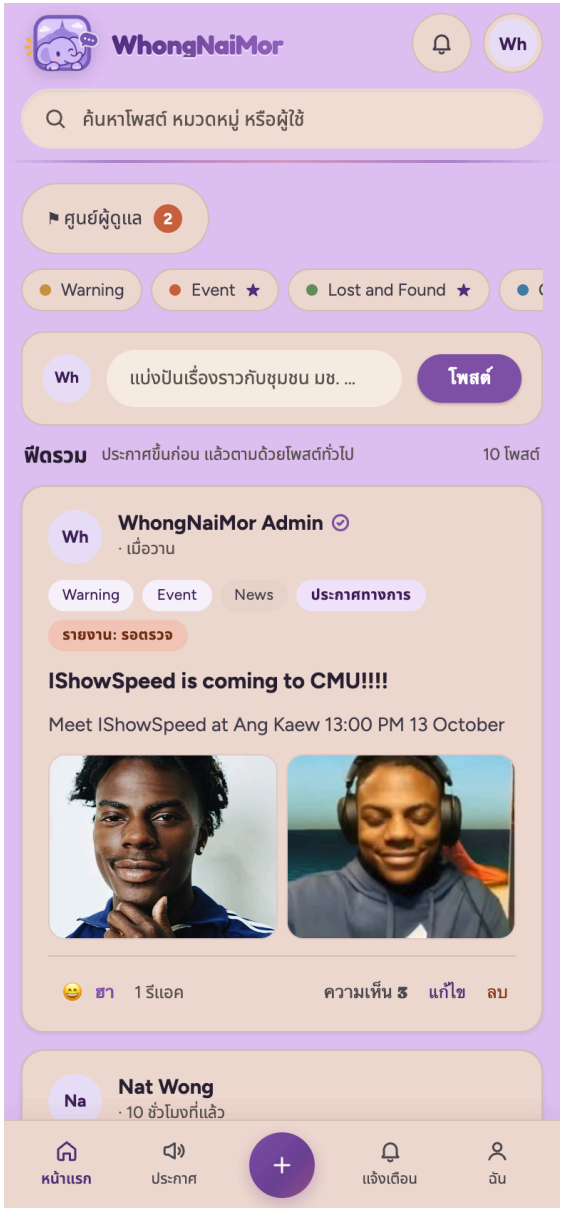
Profile

The user's own page: display name, followed categories, their posts and the reports they have made.



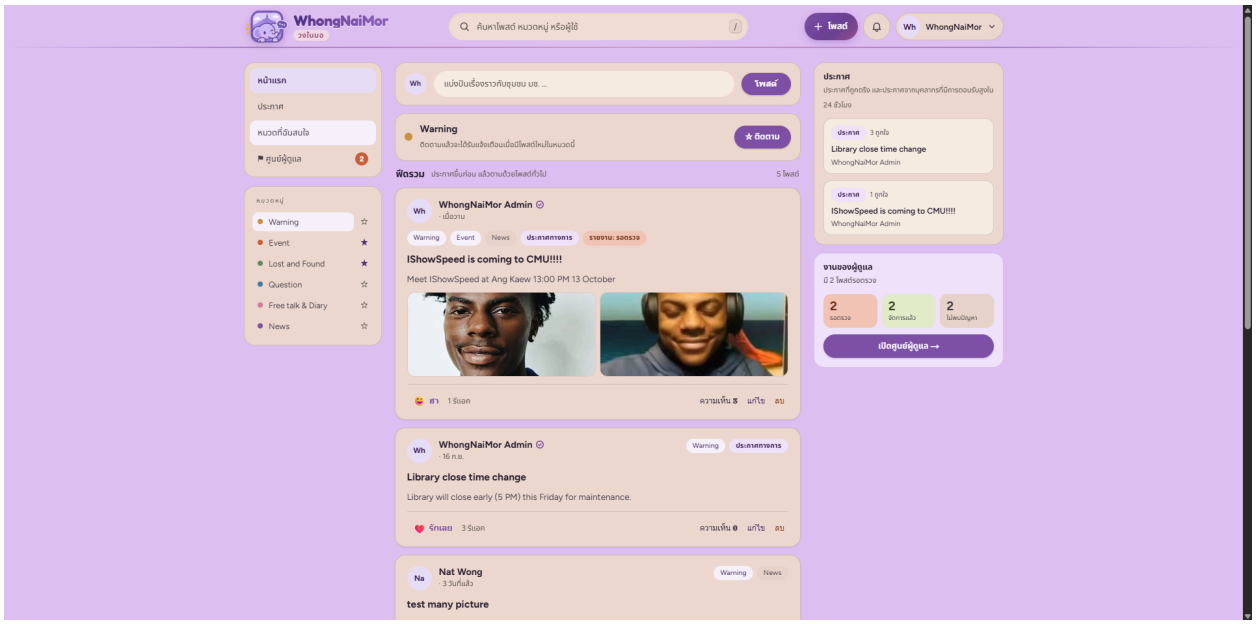
Mobile View

On a phone the side menus move into a bottom navigation bar with a button for writing a post.



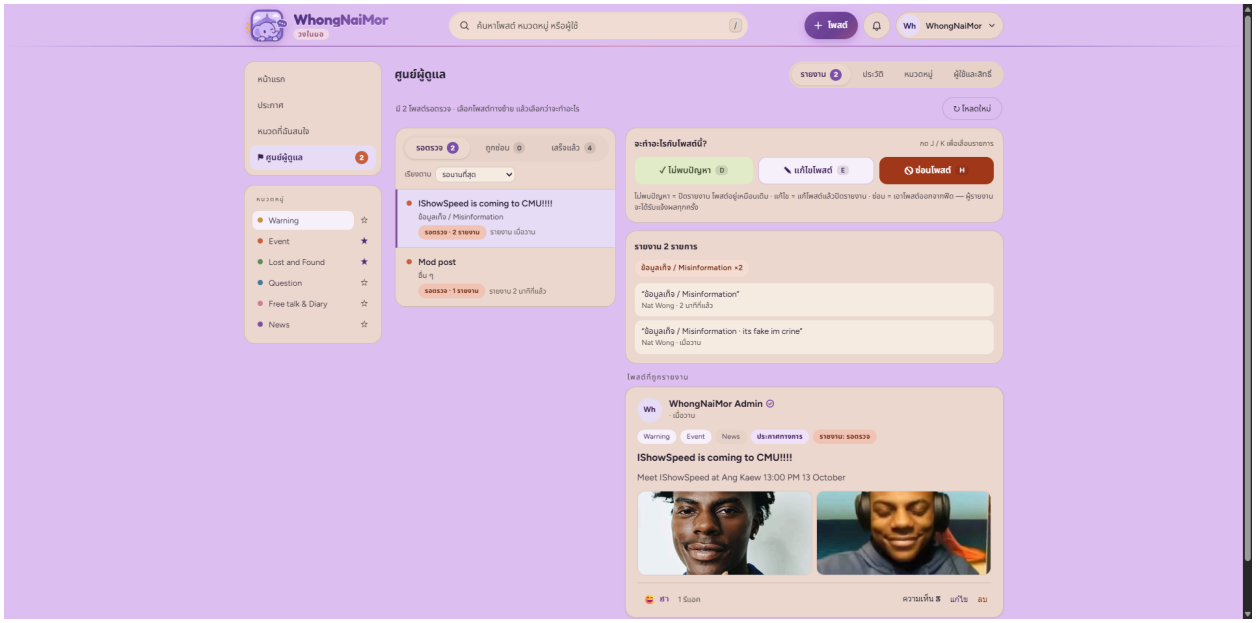
Mod View (Feed)

What a moderator sees in the feed: report badges on posts and edit and delete actions on every post.



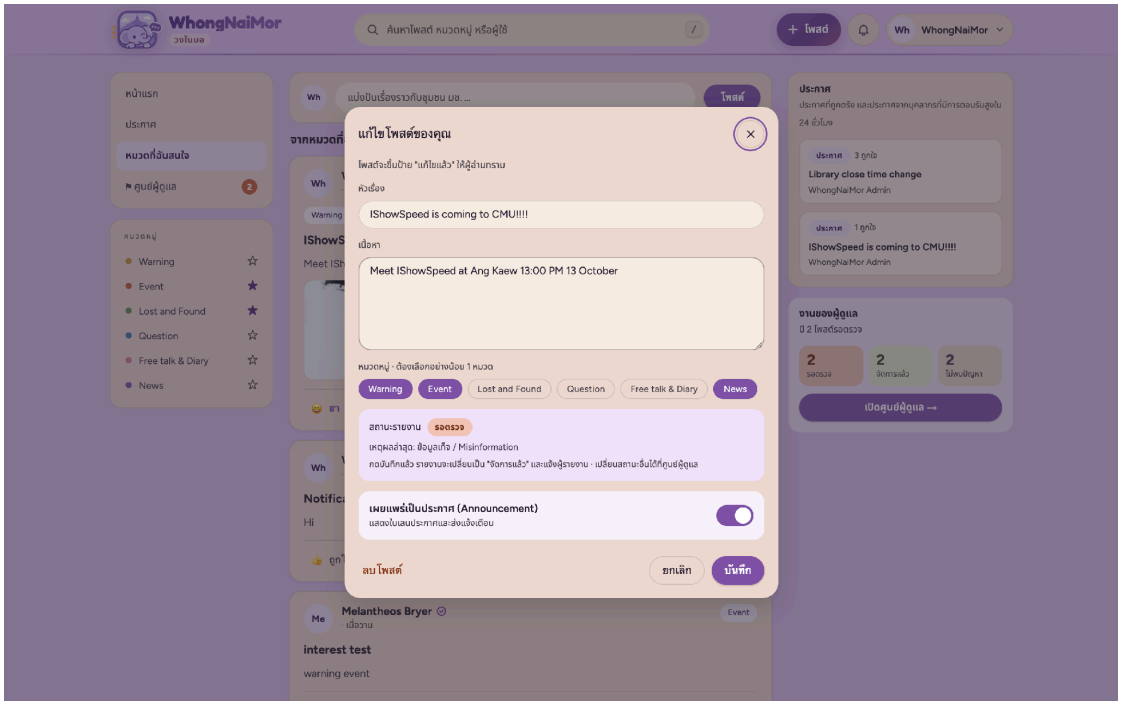
Mod Dashboard

The moderator panel on the right shows how many reports are open, actioned or dismissed.



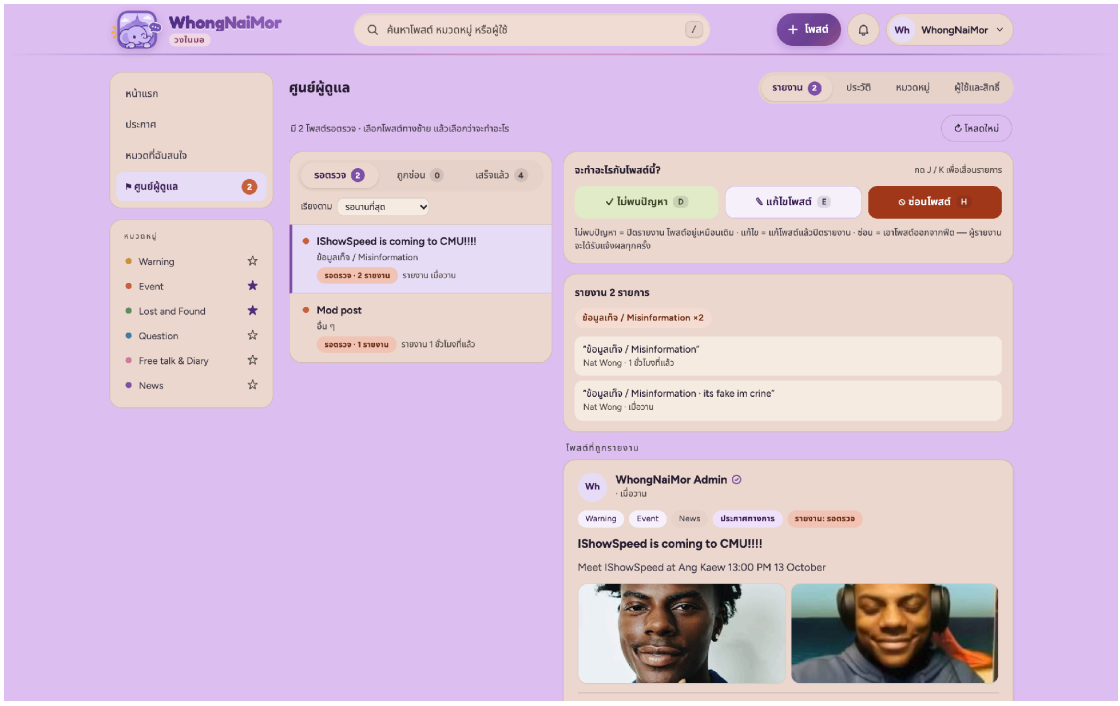
Edit Post (Moderator)

A moderator can edit any post, change its categories, turn it into an announcement, and sees its report status.



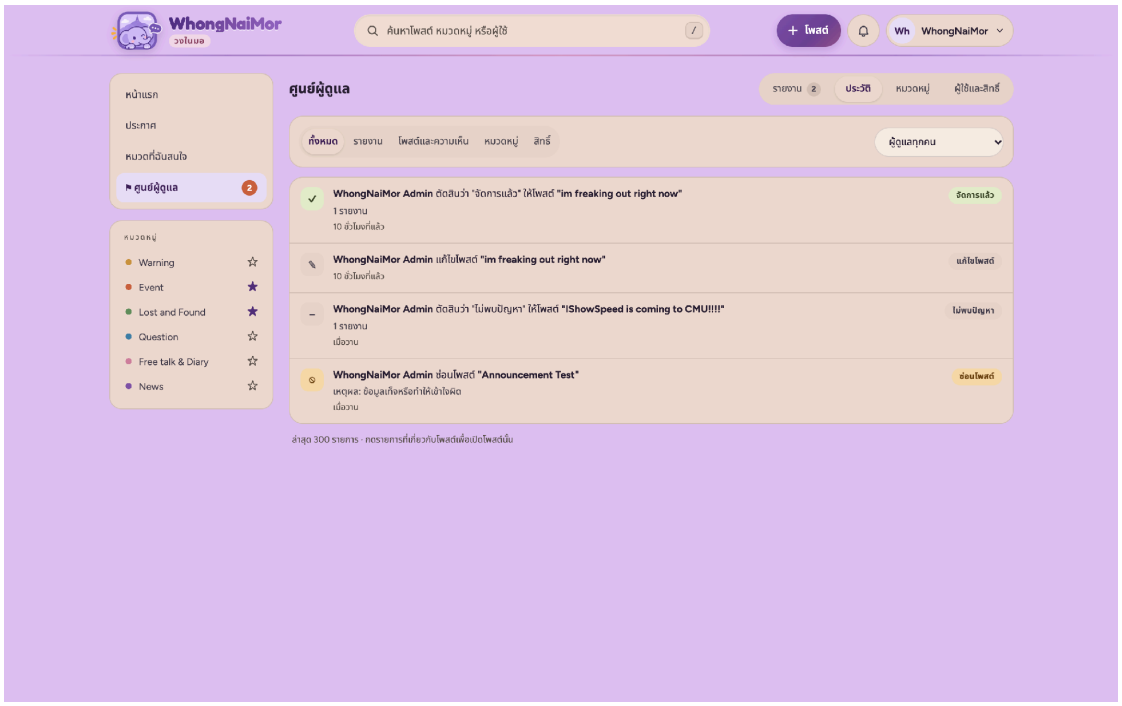
Moderation Center: Report Management

The report queue: each reported post with its reasons, and buttons to dismiss the reports, edit the post or hide it.



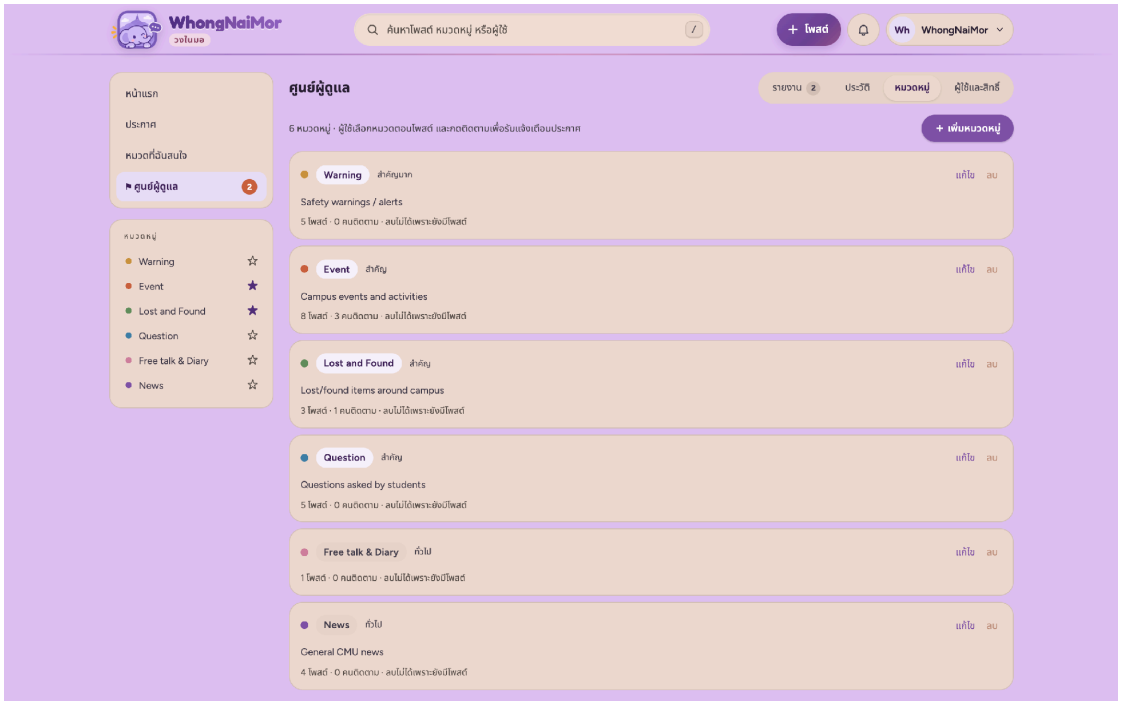
Moderation Center: History

Every moderator action with who did it and when, filterable by type and by moderator.



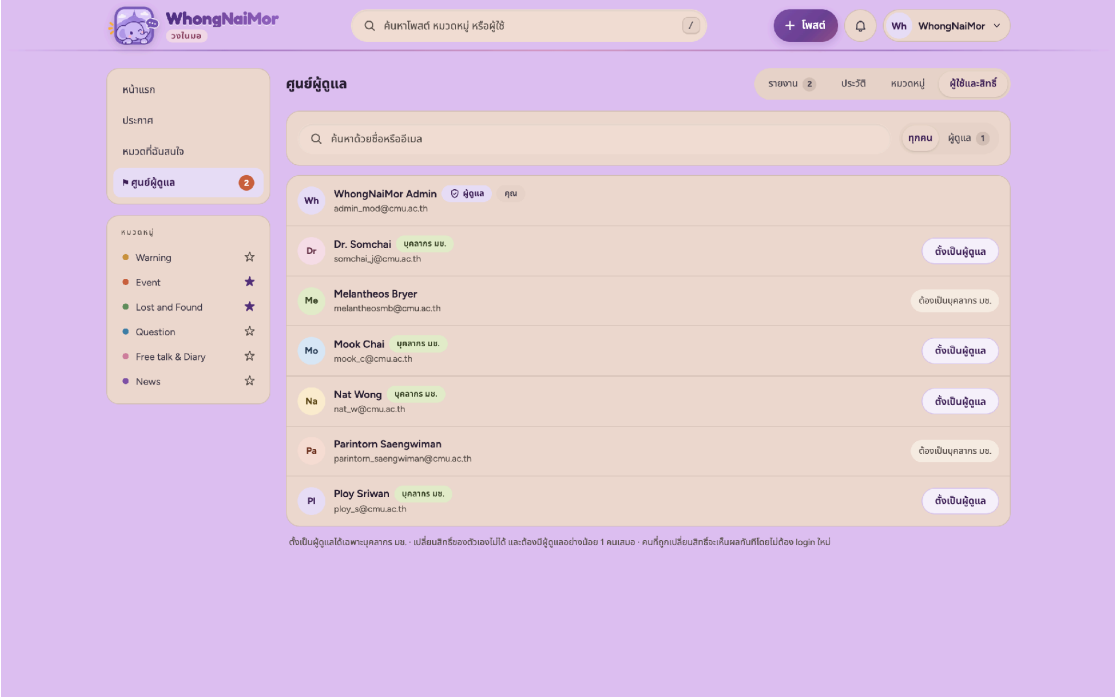
Moderation Center: Categories

Moderators add, rename and reorder categories; each shows how many posts and followers it has.



Moderation Center: Users and Roles

Moderators search users and grant or remove the moderator role; only CMU Personnel can become moderators.



Project Link

Development Repository: https://github.com/Nmpipoo/WhongNaiMor-MeTestingSoIdontExplodeMyFriendRepo
Production Repository: https://github.com/CodingWithRand/CAMT-SE/tree/WhongNaiMor
API URL: https://camt-se-0pel.onrender.com/
The Website: https://whongnaimor.vercel.app/